

Chase the Light(CTL): A Compact Post-
Quantum Key Encapsulation Mechanism Based on
NTRU Lattices and Ring-LWR

Contents

1. Introduction.....	1
1.1 Overview.....	1
1.2 Characteristic Statement	3
1.2.1 Algorithm Innovations	3
1.2.2 Algorithm Simplicity	4
1.2.3 Algorithm Flexibility	4
1.2 Document Structure	4
2. Preliminaries	5
2.1 Notation.....	7
2.2 Lattices.....	8
2.3 NTRU Problem	11
2.4 Public Key Encryption Scheme	13
2.5 Key Encapsulation Mechanism.....	15
3. Design Principle.....	16
3.1 Babai-type Decoding Algorithms Based on NTRU Lattices	16
3.1.1 Rounding Off (RO) Algorithm.....	16
3.1.2 Nearest Plane (NP) Algorithm	17
3.1.3 GGH-Micciancio Paradigm	18
3.1.4 NTRU Decoder Fusing RO and NP Algorithms.....	18
3.2 Sampling Algorithms	19
3.3 Optimization of NTRU Equation Solving	21
3.3.1 Iterative Reconstruction and Stack Space Control Design	21
3.3.2 Adaptation and Tuning of the ReducePartial Iteration Rule	22
3.3.3 Lightweight Intermediate Verification Mechanism	22
3.4 Parameter Selection	23
3.5 Principles of Engineering Implementation for Encoding, Storage and Decoding.....	25

3.5.1 Unified Design Specification for Encoding and Storage	25
3.5.2 Public Key Encoding Specification	26
3.5.3 Ciphertext Encoding Specification	27
3.5.4 Private Key Encoding Specification	28
3.5.5 Quantification of Storage Requirements.....	30
3.5.6 Decoding Principle.....	30
3.6 Description of Differences	32
4. Algorithm Description	33
4.1 Key Generation Algorithm.....	35
4.2 Encryption Sub-algorithm.....	36
4.3 Encryption Algorithm	37
4.4 Decryption Algorithm	38
4.5 Encapsulate	39
4.6 Decapsulate	39
5. Basic Components	40
5.1 Sampling Algorithm Sampler_fg	40
5.2 Auxiliary Vector Computation Algorithm ComputeVec	42
5.3 Decoding Algorithm Decode	42
6. Security Analysis	43
6.1 Security Assumption	43
6.2 Analysis with respect to known attacks	45
6.3 Decryption Failure Probability	48
7. Implementation and Performance Evaluation.....	49
7.1 Performance Analysis	49
7.2 Performance Testing	50
8. Conclusion	51
References.....	52

1. Introduction

1.1 Overview

With the rapid development of quantum computing technology, the security of traditional cryptographic systems is facing unprecedented challenges. The powerful computing capability of quantum computers poses a severe threat to widely used public-key cryptographic systems such as RSA and ECC. For instance, Shor's quantum algorithm[1] can solve the integer factorization and discrete logarithm problems in polynomial time, and Grover's quantum search algorithm[2] can reduce the search space of symmetric keys from $O(N)$ to $O(\sqrt{N})$. This means that a 128-bit AES encryption algorithm provides only about 64-bit security against quantum computers, rendering traditional security parameters unreliable. This background has driven the research and standardization process of Post-Quantum Cryptography (PQC). Since the National Institute of Standards and Technology (NIST) initiated PQC standardization in 2016, lattice-based cryptography has become one of the most competitive technical routes due to its excellent quantum-resistant security and highly efficient implementation characteristics. A typical representative is CRYSTALS-Kyber[3], which has now been officially established by NIST as a Federal Information Processing Standard named ML-KEM. The security of lattice cryptography is typically based on hard problems such as the Shortest Vector Problem (SVP) and the Closest Vector Problem (CVP), for which there are still no effective solutions under currently known quantum computing models.

In the process of PQC standardization and engineering implementation, lattice-based cryptographic schemes have gradually become the mainstream technical direction by virtue of their ability to resist quantum computing attacks and their efficient computational performance. Since the traditional NTRU cryptosystem[4] was proposed by Hoffstein, Pipher, and Silverman in 1998, its decryption process has long relied on a specific processing paradigm of "mask polynomials." Although this paradigm provides security guarantees for the cryptographic system, it limits the compactness of parameter selection to a certain extent and introduces additional boundary constraints to the decryption process.

Regarding the design of quantum-resistant schemes, lattice-based cryptography represents a class of candidate schemes with significant advantages. A lattice is a discrete additive group, and the security of lattice cryptographic systems is usually reduced to hard problems on lattices, such as the Shortest Vector Problem (SVP) and the Closest Vector Problem (CVP). The basic operations on lattices are computationally efficient, and research to date indicates that there are no effective quantum algorithms capable of rapidly solving the relevant hard problems on high-dimensional lattices. At present, the cryptographic community has developed a variety of lattice-based public-key cryptosystems.

In 1996, Ajtai[4] pioneered a reduction method for a class of problems in lattices, reducing the Shortest Independent Vectors Problem (SIVP) in the worst case to the Short Integer Solution (SIS) problem in the average case. This work laid an important foundation for the provable security of lattice cryptography. In the development of lattice-based cryptography, the hard problems associated with NTRU lattices have become an important basis for PQC design due to their data compression characteristics under a single-ring structure. The successful practical implementation of signature schemes such as Falcon[14] has verified the security of the NTRU assumption under practical parameters. Meanwhile, the introduction of the Learning with Rounding over Rings (Ring-LWR) problem has provided a new security foundation for lattice-based cryptographic design. By replacing the random error sampling process with deterministic rounding, it effectively mitigates the risk of side-channel attacks to a certain extent, while greatly promoting the efficient implementation of cryptographic schemes and the compact design of ciphertexts.

In specific constructions, NTRU-type lattices are widely employed in high-performance cryptographic design due to their compact representation and computational efficiency derived from their ring structure. Related work has shown that hard problems based on NTRU lattices and the Ring-LWR problem can provide a reliable security foundation for compact Key Encapsulation Mechanisms. Therefore, building the security of cryptographic systems on a combined model of **the decisional NTRU assumption** and **the search Ring-LWR assumption** has become a mainstream design approach that balances both security and efficiency.

This paper is designed based on the optimized NTRU lattice architecture. The

proposed scheme adopts an extremely compact parameter system. Under the same security level, it can utilize a smaller computational modulus to greatly reduce the algorithm implementation complexity. Meanwhile, it delivers ultra-small public key and ciphertext sizes, remarkably reduces data storage consumption, achieves extremely low communication bandwidth occupancy, and effectively cuts data transmission costs as well as the risk of packet loss. On this basis, targeted optimization and improvement are conducted to address the defects of existing schemes [6], such as low efficiency, poor stability and incomplete coverage of security levels in trapdoor generation, equation solving and parameter configuration.

1.2 Characteristic Statement

1.2.1 Algorithm Innovations

(1) High-quality (f, g) Generation Method Based on Hybrid Sampler

Traditional NTRU trapdoor generation adopts the inefficient workflow of independent sampling over coefficient domain combined with posterior screening. This paper introduces a hybrid sampler for NTRU lattices [7] to constrain trapdoor quality directly from the perspective of distribution design. Specifically, leveraging the Fourier embedding structure of cyclotomic rings, we perform approximately uniform sampling in the frequency domain and adopt a rounding error compensation mechanism. The sampled results inherently satisfy the geometric constraints of trapdoor bases.

This method boasts three prominent merits: it eliminates a large number of invalid samples and raises the sampling success rate; it mitigates fluctuations in trapdoor quality and enhances algorithm stability; it supports flexible parameter configuration and reserves room for subsequent optimization.

(2) Decision Criteria and Workflow Optimization for NTRU Equation Solving

Solving the NTRU equation constitutes the major computational bottleneck in the key generation phase. On the basis of improved sampling quality of (f, g) , this paper proposes pre-judgment criteria for validity to eliminate infeasible samples prior to equation solving. Meanwhile, an adaptive iteration termination strategy is formulated to avoid redundant iterative loops, cut down the waste of computing resources, and improve the overall efficiency of key generation.

(3) Iterative Strategy and Algorithm Optimization for NTRU Equation

To address the drawbacks of conventional NTRU equation solving, including low iteration efficiency, high stack space occupancy and uncontrollable failure rate, this paper introduces relevant iterative optimization methods [8] and adapts them to the application scenarios of the proposed scheme, so as to optimize the solving performance and operational stability of the NTRU equation.

The ReducePartial iterative optimization strategy is adopted to strike a balance among iteration times, solving failure rate and running speed, thereby reducing the overall solving failure rate. A lightweight intermediate iteration verification mechanism is further designed to match the characteristics of ReducePartial iteration. It detects invalid iterations at low computational cost and terminates the process in advance, which reduces the overhead caused by useless computations.

1.2.2 Algorithm Simplicity

Plaintext recovery in the decryption process is implemented via a small set of fundamental algebraic operations. The corresponding program features concise code and clear logical hierarchy, facilitating algorithm verification, programming implementation and subsequent maintenance.

The public key and ciphertext adopt simple encoding formats, which lowers the cost of data storage and parsing.

1.2.3 Algorithm Flexibility

This scheme establishes a complete parameter set system covering multi-level security strengths. Optimal parameter configurations can be flexibly selected according to security requirements, terminal computing power constraints and communication bandwidth limitations in diverse application scenarios. This design takes security demands, performance overhead and communication efficiency into comprehensive consideration. With strong universality, the algorithm is applicable to a wide range of scenarios.

1.2 Document Structure

Chapter 1: Introduction. This chapter elaborates on the research background, current industry status, existing problems, core optimization directions, and main technical contributions of the proposed scheme, while also outlining the overall

structure of the document.

Chapter 2: Preliminaries. This chapter unifies the mathematical notations used throughout the text and reviews the fundamental theory of lattice-based cryptography, main NTRU problems, formal definitions of Public-Key Encryption (PKE) and Key Encapsulation Mechanisms (KEM), and their security semantics, establishing the theoretical framework for the scheme's design.

Chapter 3: Algorithmic Strategy. This chapter details the core design methodology of the scheme, covering key modules such as sampling mechanisms, the parameter system, decoding algorithms, encoding and storage specifications, and the engineering implementation of the decoding process.

Chapter 4: Algorithm Description. This chapter completely defines the core algorithms for the entire process of key generation, encryption/decryption, and encapsulation/decapsulation of the scheme using pseudocode.

Chapter 5: Fundamental Components. This chapter breaks down and standardizes the implementation details of the underlying supporting algorithms upon which the core algorithms rely, ensuring the reproducibility of the scheme.

Chapter 6: Security Analysis. This chapter clarifies the core security assumptions the scheme relies upon, completes the theoretical security reduction proofs, and verifies the practical security strength of the scheme against mainstream attacks.

Chapter 7: Implementation and Performance Testing. This chapter completes the quantitative testing and evaluation of the core performance metrics.

Chapter 8: Conclusion. This chapter summarizes the core work and primary innovations of the entire document.

2. Preliminaries

In this paper, we define the polynomial ring $R = \mathbb{Z}[x]/(x^n + 1)$, where n is a power of 2, the elements in the ring are integer-coefficient polynomials with a degree less than n , the addition and multiplication between polynomials are ring operations modulo $x^n + 1$; the residue class ring modulo q , denoted as R/qR , represents the set of operation results where the coefficients of elements in ring R are modulo a positive integer q . We define φ as the canonical embedding mapping, which maps elements in ring R to the complex vector space in the Fourier domain, and φ^{-1} as its inverse

mapping, realizing the inverse transformation from complex vectors in the Fourier domain to the polynomial ring R .

Canonical embedding is a homomorphic mechanism that maps algebraic elements in a cyclotomic field to high-dimensional complex vectors. The cyclotomic field $\mathcal{K}$ has d complex field embeddings $\varphi_i: \mathcal{K} \rightarrow \mathbb{C}$. When treating an element f in the cyclotomic field as a polynomial, these embeddings are actually equivalent to evaluating the polynomial at ζ^k , where k satisfies $\gcd(k, m) = 1$, and m is the order of the cyclotomic polynomial. The aforementioned d complex field embeddings jointly define the canonical embedding, which is formalized as a vector containing the evaluations of the element under all these homomorphisms, i.e., $\varphi(f) := (\varphi_1(f), \dots, \varphi_d(f))$. This mapping mechanism not only strictly preserves the multiplicative homomorphic property over the ring, allowing it to be extended term-by-term and applied to higher-dimensional vectors or underlying private key matrix operations, but also geometrically and topologically allows the continuous extended space to be naturally equated to a specific conjugate-symmetric complex vector space $\mathbb{H}$. Specifically, in an encryption system based on a power of 2 cyclotomic ring, this embedding manifests as an evaluation mapping at the $2n$ -th primitive roots of unity in the complex plane.

Define three categories of parameters: security parameters, trapdoor parameters, and encryption/decryption parameters.

The security parameters include the positive integer n , representing the polynomial ring dimension; the positive integer q , representing the modulus; and the positive integer k , representing the block parameter, which is used for *Ring-LWR* rounding and ciphertext compression.

The trapdoor parameters include the positive real number α , representing the trapdoor quality parameter, which is used to control the trapdoor validity of the short polynomial pair (f, g) ; and the positive real number ξ , $0 < \xi < 1$, representing the error compensation factor, which is used to offset the errors from sampling and rounding, this paper sets it to $1/3$.

The encryption and decryption parameters include the positive integer q' , representing the decryption failure rate control parameter; the positive real number γ , representing the distribution balance parameter, which is used to adapt to the

distribution differences between the message and the error term; and the positive integer Q , representing the decoding scale parameter to ensure integer operations, this paper takes 2.

The security of the scheme proposed in this paper is based on the following two computational hardness assumptions:

Decisional NTRU Assumption: For the polynomial ring R and modulus q , the public key $h = f^{-1}g \bmod q$ generated by the short polynomial pair (f, g) is computationally indistinguishable from an element chosen uniformly at random from the ring R/qR .

Search Ring-LWR Assumption: Given multiple sets of samples $\{(a_i, b_i)\}_{i=1}^m$, where $a \in R/qR$ is a uniformly random element and $b_i = \lfloor (p/q)(a_i \cdot s) \rfloor \in R/pR$, there is no polynomial-time algorithm that can recover the corresponding secret polynomial s from these samples.

2.1 Notation

The basic notations used in this paper are as follows.

Lowercase letters $a, b, \dots$ denote elements in the ring R or R_q (including $\mathbb{Z}$ and $\mathbb{Z}_q$).

Bold lowercase letters $\mathbf{a}, \mathbf{b}, \dots$ denote column vectors with elements belonging to R or R_q (including $\mathbb{Z}$ and $\mathbb{Z}_q$).

Bold uppercase letters $\mathbf{A}, \mathbf{B}, \dots$ denote matrices with elements belonging to R or R_q (including $\mathbb{Z}$ and $\mathbb{Z}_q$).

For an n -dimensional vector v , the i -th element is denoted as $v_i (0 \leq i < n)$.

$\mathbf{A}^{m \times n}$ denotes an $m \times n$ matrix.

$(\mathbf{u}, \mathbf{v}), \begin{pmatrix} \mathbf{u} \\ \mathbf{v} \end{pmatrix}, (\mathbf{u}|\mathbf{v})$ denote the concatenation of vectors, and $(\mathbf{A}|\mathbf{b})$ denotes matrix concatenation.

For an $m \times n$ matrix A , the (i, j) -th entry, i.e., the element in the i -th row and j -th column, is denoted as $A_{i,j} (0 \leq i < m, 0 \leq j < n)$.

$\langle \mathbf{s}, \mathbf{a} \rangle$ denotes the inner product of vectors $\mathbf{s}$ and $\mathbf{a}$. For example, for $\mathbf{s} = (s_1, s_2, \dots, s_n)^T \in \mathbb{Z}_q^n$ and $\mathbf{a} = (a_1, a_2, \dots, a_n)^T \in \mathbb{Z}_q^n$, have $\langle \mathbf{s}, \mathbf{a} \rangle = s_1 a_1 + s_2 a_2 + \dots + s_n a_n \in \mathbb{Z}_q$.

$\bar{f}$ denotes the conjugate of f , $\overline{f(x)} = f(x^{-1})$

$\lfloor a \rfloor$ denotes rounding a down to the nearest integer.

$\lceil a \rceil$ denotes rounding a up to the nearest integer.

$\lceil a \rceil$ denotes rounding a to the nearest integer.

$[\cdot]_q$ denotes taking the result modulo q and mapping it to the symmetric interval of $\mathbb{Z}_q$.

The set of integers is denoted by $\mathbb{Z}$.

The set of positive integers is denoted by $\mathbb{N}$.

The set of rational numbers is denoted by $\mathbb{Q}$.

The set of integers modulo q is denoted by $\mathbb{Z}_q$, i.e., $\{-(\lfloor q/2 \rfloor - 1), -(\lfloor q/2 \rfloor - 2), \dots, q - \lfloor q/2 \rfloor\}$.

The set of all polynomials with integer coefficients is denoted by $\mathbb{Z}[x]$.

The polynomial ring $\mathbb{Z}[x]/f(x)$ is denoted by R_n , where $f(x) = x^n + 1$.

The polynomial ring over the rational field $\mathbb{Q}[x]/f(x)$ is denoted by $\mathbb{K}_n$, where $f(x) = x^n + 1$

The set of invertible elements in R/qR is denoted by $R_q^\times$.

2.2 Lattices

Lattice is a mathematical structure that originated from Gauss's research on quadratic forms and was systematically developed by Minkowski. A lattice is an integer-coefficient linear combination of n linearly independent non-zero vectors in an m -dimensional Euclidean space $\mathbb{R}^m$. Let $b_1, \dots, b_n \in \mathbb{R}^m$ be a set of linearly independent vectors, the set generated by the linear combination of the vector $b_1, \dots, b_n$ is called a lattice in $\mathbb{R}^m$, namely

$$\mathcal{L}(b_1, \dots, b_n) = \left\{ \sum_{i=1}^n x_i b_i : x_i \in \mathbb{Z} \right\}$$

The vectors $b_1, \dots, b_n$ are considered a basis of the lattice, m is called the

dimension of the ambient space, and n is called the dimension (or rank) of the lattice. If $m = n$, the lattice is said to be full-rank. A basis of a lattice can also be represented in matrix form. By taking $b_1, \dots, b_n$ as column vectors to form the matrix $B = [b_1, \dots, b_n] \in \mathbb{R}^{m \times n}$, the lattice L can be represented as

$$\mathcal{L}(B) = \{Bx : x \in \mathbb{Z}^n\}$$

The lattice basis for a lattice $\mathcal{L} = \mathcal{L}(B)$ is not unique, an n -dimensional lattice ($n \geq 2$) has infinitely many bases. For any given lattice basis B , the fundamental domain of the lattice $\mathcal{L} = \mathcal{L}(B)$ is defined as the parallelepiped spanned by the n basis vectors. Its volume is referred to as the volume of the lattice $\mathcal{L}$, denoted as $\text{vol}(\mathcal{L})$ or the determinant, denoted as $\det(\mathcal{L})$. The determinant is $\det(\mathcal{L}) = \sqrt{\det(B^T B)}$, and its value is independent of the choice of the basis B , remaining constant. The dual lattice $\mathcal{L}^*$ is defined as

$$\mathcal{L}^* = \{\mathbf{w} \in \mathbb{R}^m \mid \forall \mathbf{v} \in \mathcal{L}, \langle \mathbf{w}, \mathbf{v} \rangle \in \mathbb{Z}\}$$

The definition of a lattice is similar to, but slightly different from, the mathematical span, which is defined as follows.

$$\text{span}(B) = \left\{ \sum_{i=1}^n x_i b_i : x_i \in \mathbb{R} \right\}$$

Similar to the definition of a subgroup, the definition of a sublattice is: if $\mathcal{L} \in i^m$ is a lattice, then $\mathcal{L}' \subset \mathcal{L}$ is a sublattice of $\mathcal{L}$.

Here is another equivalent definition of a lattice: a lattice is a discrete additive subgroup over i^m , i.e. a lattice is a subset $\Lambda \subseteq i^m$ that satisfies the following two properties

- (1) Λ is closed under addition and subtraction;
- (2) There exists a number $\varepsilon > 0$ such that the distance between any two distinct lattice points $x \neq y \in \Lambda$ is at least $\|x - y\| > \varepsilon$.

here are two most fundamental hard problems in lattices, namely the Shortest Vector Problem (SVP) and the Closest Vector Problem (CVP).

Definition 2-1 (SVP Problem) : Given a lattice basis B , find a shortest non-zero vector $\mathbf{v}$ in the lattice $\mathcal{L}(B)$, such that for any non-zero vector $u \in \mathcal{L}(B)$, it holds that $\|\mathbf{v}\| \leq \|u\|$.

Definition 2-2 (CVP Problem) Given a lattice basis B and a target vector t that

is not in the lattice $\mathcal{L}(B)$, find a lattice vector $v \in \mathcal{L}(B)$ such that for any vector $u \in \mathcal{L}$ and, it holds that $\|v - t\| \leq \|u - t\|$.

There are also some variants of SVP and CVP.

Definition 2-3 (Optimization SVP) Given a lattice basis B , find the first successive minimum $\lambda_1(\mathcal{L}(B))$ in the lattice.

Definition 2-4 (Decisional SVP) Given a lattice basis B and a rational number r , decide whether the relation $\lambda_1(\mathcal{L}(B)) \leq r$ holds.

Definition 2-5 (γ -Approximate Shortest Vector Problem, SVP- γ) Given a lattice $\mathcal{L}(B)$, find a non-zero lattice vector v satisfying $\|v\| \leq \gamma \cdot \lambda_1(\mathcal{L}(B))$.

Definition 2-6 (Optimization SVP- γ) Given a lattice basis B , find a value d such that $d \leq \lambda_1(\mathcal{L}(B)) \leq \gamma d$.

Definition 2-7 (Decisional γ -Approximate Shortest Vector Problem, GapSVP- γ) Given a lattice basis B and a rational number r , decide whether the instance (B, r) is a YES instance ($\lambda_1(\mathcal{L}(B)) \leq r$) or a NO instance ($\lambda_1(\mathcal{L}(B)) > \gamma r$). The algorithm's behavior is undefined in the intermediate gap region.

Definition 2-8 (Shortest Independent Vectors Problem, SIVP) Given a lattice $\mathcal{L}(B)$ of rank n , find n linearly independent lattice vectors s_i satisfying $\max \|s_i\| \leq \lambda_n(\mathcal{L}(B)), i = 1, \dots, n$.

Definition 2-9 (Successive Minima Problem, SMP) Given a lattice $\mathcal{L}(B)$ of rank n , find n linearly independent lattice vectors s_i satisfying $\lambda_i(\mathcal{L}(B)) = \|s_i\|, i = 1, \dots, n$.

Definition 2-10 (Unique Shortest Vector Problem, uSVP- γ) Given a lattice $\mathcal{L}$ satisfying $\lambda_2(\mathcal{L}) > \gamma \lambda_1(\mathcal{L})$, find the unique shortest vector of the lattice.

Definition 2-11 (Optimization CVP) Given a lattice $\mathcal{L}$ and a target vector t , find a value d satisfying $d \leq \text{dist}(t, \mathcal{L}(B)) \leq \gamma d$.

Definition 2-12 (GapCVP- γ Problem) Given a lattice basis B , a rational number r , and a target vector t , decide whether the instance (B, r, t) is a YES instance ($\text{dist}(t, \mathcal{L}(B)) \leq r$) or a NO instance ($\text{dist}(t, \mathcal{L}(B)) > \gamma r$).

Definition 2-13 (Bounded Distance Decoding, BDD) Given a lattice $\mathcal{L}$ and a

target vector $t \in \mathbb{R}^n$, find the unique lattice point v satisfying $\|v - t\| < \lambda_1(\mathcal{L})/2$, where $v \in \mathcal{L}$ and $\lambda_1(\mathcal{L})$ is the length of the shortest non-zero vector in lattice $\mathcal{L}$.

The underlying implementation of the scheme in this paper reduces to the BDD problem on NTRU lattices.

2.3 NTRU Problem

In lattice-based cryptographic systems, NTRU is one of the oldest and most efficient practical schemes, proposed by Hoffstein et al. [4] in 1996. To perform cryptanalysis on the NTRU system, Coppersmith and Shamir[9] first mapped the polynomial algebraic relations of NTRU into a high-dimensional geometric space in 1997, this specific geometric structure is known as the NTRU lattice. The core idea of NTRU is to utilize a mix of algebraic inversion and multiplication in polynomial rings to construct a one-way trapdoor. Its security is based on the Shortest Vector Problem (SVP) in the NTRU lattice, which is considered a mathematically hard problem and currently lacks any algorithm capable of solving it in polynomial time. The NTRU algorithm is believed to be resistant to quantum attacks and has passed the standardization certification of the Institute of Electrical and Electronics Engineers (IEEE).

The core parameters of NTRU include: a prime number n , and two coprime moduli p and q . n represents the highest degree of the polynomials; typically, $p = 3$, and q is a relatively large number. All operations are performed in the truncated polynomial ring $R_q = \mathbb{Z}_q[X]/(X^n - 1)$. Two polynomials with extremely small coefficients, $f(x)$ and $g(x)$, are selected in the ring R_q . It is required that $f(x)$ has an inverse in both the modulo p and modulo q rings. The private key is exactly this pair of polynomials with extremely small coefficients, (f, g) .

The public key $h(x)$ is obtained by computing the product of the inverse of $f(x)$ modulo q and $g(x)$

$$h(x) \equiv f_q^{-1}(x) \cdot g(x) \pmod{q}$$

This implies that in the polynomial ring R_q , the following algebraic identity holds

$$f(x) \cdot h(x) \equiv g(x) \pmod{q}$$

It is further transformed into an equation without modular operations; there exists a certain polynomial $u(x)$ such that

$$f(x) \cdot h(x) - q \cdot u(x) = g(x)$$

To transform the above polynomial relationship into the Shortest Vector Problem on a lattice, polynomial multiplication is represented as matrix multiplication. For a polynomial $a(x) = \sum_{i=0}^{N-1} a_i x^i$, its coefficients can be extracted to represent the polynomial as an N -dimensional row vector $\mathbf{a} = (a_0, a_1, \dots, a_{N-1})$. In the ring $R_q = \mathbb{Z}[X]/(X^N - 1)$, multiplying by a polynomial $h(x)$ is equivalent to multiplying by an $N \times N$ circulant matrix $\mathbf{H}$ formed by cyclic shifts of the coefficients of $h(x)$

$$\mathbf{H} = \begin{pmatrix} h_0 & h_1 & \cdots & h_{N-1} \\ h_{N-1} & h_0 & \cdots & h_{N-2} \\ \vdots & \vdots & \ddots & \vdots \\ h_1 & h_2 & \cdots & h_0 \end{pmatrix}$$

the equation without modular operations can be strictly mapped to vector and matrix multiplication

$$\mathbf{f}\mathbf{H} - \mathbf{q}\mathbf{u} = \mathbf{g}$$

To unify all unknown vectors on both sides of the equation into the same geometric space, Coppersmith and Shamir constructed a $2N$ -dimensional block matrix $\mathbf{M}$

$$\mathbf{M} = \begin{pmatrix} q\mathbf{I}_N & \mathbf{0}_N \\ \mathbf{H} & \mathbf{I}_N \end{pmatrix}$$

Where $\mathbf{I}_N$ is the $N \times N$ identity matrix, and $\mathbf{0}_N$ is the $N \times N$ zero matrix.

The $2N$ -dimensional lattice $\mathcal{L}(\mathbf{M})$ generated by the linear combinations of the column vectors of matrix $\mathbf{M}$ over the integer ring $\mathbb{Z}$ is exactly the NTRU lattice.

In the first decade or so, the NTRU assumption primarily relied on empirical security, it lacked a rigorous worst-case to average-case mathematical reduction like LWE. In 2011, Stehlé and Steinfeld proposed[10] that if, when generating the private key polynomials f and g , one samples from a discrete Gaussian distribution with a slightly larger variance instead of using strictly minimal fixed coefficients, the generated NTRU public key h is statistically close to a uniform distribution. This discovery successfully reduced the security of NTRU to the problem of solving the unique Shortest Vector Problem in Ideal Lattices.

Since the NTRU lattice basis is a lower triangular block matrix, its determinant is easily obtained

$$\det(\mathcal{L}) = \det(q\mathbf{I}_N) = q^N$$

According to the Gaussian heuristic, the expected length of the shortest non-zero

vector in the NTRU lattice is approximately

$$\lambda_1(\mathcal{L}) \approx \sqrt{\frac{2N}{2\pi e}} \cdot \det(\mathcal{L})^{\frac{1}{2N}} = \sqrt{\frac{N}{\pi e}} \cdot \sqrt{q}$$

It follows from $\mathbf{fH} - \mathbf{qu} = \mathbf{g}$ that the vector (f, g) satisfies the linear relation in the NTRU lattice, which indicates that the private keys f, g indeed lie within the NTRU lattice. Since the coefficients of f and g are extremely small, the norm of v , i.e., $\|v\|$ is also extremely small, much smaller than the theoretical predicted value of the Gaussian heuristic. Obtaining the private key v is equivalent to solving the unique Shortest Vector Problem in a $2n$ -dimensional special ideal lattice.

Traditional NTRU is established on the truncated polynomial ring $X^n - 1$, which is highly susceptible to algebraic dimension-reduction attacks. In pursuit of polynomial irreducibility, the high efficiency of Number Theoretic Transform (NTT) acceleration, and better geometric error control, modern mainstream schemes based on the NTRU architecture, such as Falcon, have fully migrated to the cyclotomic ring of $X^n + 1$, where n is a power of 2. The cryptographic system constructed in this paper is established on the cyclotomic ring of $X^n + 1$. It abandons the decryption mechanism of traditional NTRU, which relies on mask parameters and double modular reduction, and transforms the decryption process into a pure multi-dimensional lattice space decoding problem.

2.4 Public Key Encryption Scheme

Definition 2-14 (Formal Syntax) : A standard public key encryption scheme PKE consists of three polynomial-time algorithms: $KeyGen$, Enc , Dec , and a plaintext space $\mathcal{M}$.

Key Generation Algorithm $KeyGen(1^\lambda)$

Input: The system security parameter λ , passed in unary form 1^λ , used to bound the theoretical lower limit of computational complexity.

Output: Generates an asymmetric key pair (pk, sk) , where the public key pk is used for public distribution, and the secret key sk is securely retained by the receiver. This algorithm is typically a probabilistic polynomial-time algorithm.

Encryption Algorithm $Enc(pk, m)$

Input: The receiver's public key pk and the message m to be encrypted,

defined within the plaintext space $\mathcal{M}$.

Output: Generates a ciphertext $c \in \mathcal{C}$. To resist replay attacks and statistical analysis, this algorithm must be probabilistic, meaning that multiple encryptions of the same plaintext must produce varying ciphertexts with a pseudorandom distribution.

Decryption Algorithm $Dec(sk, c)$

Input: The receiver's secret key sk and the ciphertext c .

Output: Recovers the original plaintext m . If the ciphertext structure is invalid or has been tampered with, it outputs a rejection symbol $\perp$. This algorithm is typically a deterministic polynomial-time algorithm.

Correctness Condition For any key pair (pk, sk) legitimately generated by $KeyGen$ and any plaintext $m \in \mathcal{M}$, the decryption algorithm must execute successfully with an overwhelmingly high probability

$$Pr[Dec(sk, Enc(pk, m)) = m] \geq 1 - negl(\lambda)$$

Where $negl(\lambda)$ represents a negligible function that decays rapidly as the security parameter λ increases.

Definition 2-15 (Security Semantics) Let PKE be a public-key encryption scheme and $\mathcal{A}$ be a polynomial-time adversary. This security semantic is strictly equivalent to indistinguishability under chosen-plaintext attack (IND-CPA), which is quantitatively verified through an interactive experiment defined between a challenger and $\mathcal{A}$:

- (1) The challenger runs $KeyGen(1^\lambda)$ to generate (pk, sk) , and hands over the public key pk to the adversary $\mathcal{A}$;
- (2) The adversary $\mathcal{A}$ freely constructs and submits two plaintexts of equal length, (m_0, m_1) ;
- (3) The challenger internally performs a uniformly random coin toss to obtain a secret bit $b \in \{0, 1\}$;
- (4) The challenger computes the challenge ciphertext $c^* = Enc(pk, m_b)$ and returns the result to the adversary $\mathcal{A}$;
- (5) Using the public key pk , $\mathcal{A}$ continues to perform arbitrary encryption deductions within polynomial time and outputs a guessed value b' for b .

Security Definition: If the encryption scheme possesses IND-CPA security semantics, it requires that for any polynomial-time adversary $\mathcal{A}$, its advantage in

winning the aforementioned game (i.e., $b' = b$) compared to random guessing must be negligible

$$Adv_{\mathcal{A}, pke}^{ina-cpa}(\lambda) = \left| Pr[b' = b] - \frac{1}{2} \right| \leq negl(\lambda)$$

In practical industrial applications and post-quantum cryptography deployment scenarios, since attackers can actively send maliciously constructed ciphertexts to decryption servers, the IND-CPA model is no longer sufficient to cover all threats. Therefore, modern standard specifications further require schemes to satisfy indistinguishability under adaptive chosen-ciphertext attack (IND-CCA). In the IND-CCA model, the adversary is granted even more extreme privileges: after obtaining the challenge ciphertext c^* , the adversary is still allowed to submit arbitrary ciphertexts to the system's decryption oracle and obtain their decryption results, but is strictly prohibited from directly submitting c^* itself. If the scheme can still satisfy the aforementioned advantage formula under such high-pressure attacks, it can then be recognized as possessing the highest level of security semantics required for deployment in complex networks.

2.5 Key Encapsulation Mechanism

Definition 2-16 (Formal Syntax): A standard Key Encapsulation Mechanism Π_{KEM} consists of three probabilistic polynomial-time algorithms: *KeyGen*, *Encaps* and *Decaps*, together with a key space $\mathcal{K}$ defined over the security parameter λ .

Key Generation Algorithm *KeyGen*(1^λ)

Input: The system security parameter λ , imported in unary form 1^λ , which defines the theoretical lower bound of computational complexity.

Output: An asymmetric key pair (pk, sk) , where the public key pk is for public distribution and the secret key sk is securely retained by the receiver. This algorithm is generally a probabilistic polynomial-time algorithm.

Encapsulation Algorithm *Encaps*(pk)

Input: The receiver's public key pk .

Output: A uniformly random symmetric key $k \in \mathcal{K}$ generated internally, along with the corresponding encapsulated ciphertext c . This algorithm must be a probabilistic polynomial-time algorithm to guarantee high entropy and unpredictability of the keys and ciphertexts generated in each execution.

Decapsulation Algorithm $Decaps(sk, c)$

Input: The receiver's secret key sk and the ciphertext c .

Output: The recovered encapsulated symmetric key k . If the ciphertext is malformed or tampered with, the algorithm outputs the rejection symbol $\perp$. This algorithm is generally a deterministic polynomial-time algorithm.

Correctness Condition

For any valid key pair (pk, sk) generated by $KeyGen$, and any ciphertext-key pair (c, k) output by $Encaps$, the decapsulation algorithm shall correctly recover the key.

$$Pr[Decaps(sk, c) = k] = 1$$

Some lattice-based cryptography schemes permit a negligible decapsulation failure probability.

3. Design Principle

3.1 Babai-type Decoding Algorithms Based on NTRU Lattices

To achieve efficient and reliable decryption of ciphertexts in our scheme, the decoding module employs a Babai-type approximate closest vector algorithm (Approximate CVP Solver) based on NTRU trapdoors. Under the premise of utilizing the private key trapdoor basis, this algorithm can recover the message and error components corresponding to the target vector with relatively low computational complexity. Let the public key of the NTRU cryptosystem be $h \in (\mathcal{R} \bmod q)$, where the polynomial ring is $\mathcal{R} = \mathbb{Z}[x]/(x^n + 1)$, and q is the large modulus of the cryptosystem; the corresponding private key trapdoor basis is defined as $B_{f,g} = \begin{pmatrix} g & G \\ f & F \end{pmatrix} \in \mathcal{R}^{2 \times 2}$, which spans the NTRU lattice $\mathcal{L}_{h,q}$. In the implementation and analysis, the $\mathcal{R}$ -module representation is adopted to describe the structure. This representation is equivalent to the standard $2n$ -dimensional integer embedding but is more conducive to characterizing the polynomial convolution structure and the actual implementation process.

3.1.1 Rounding Off (RO) Algorithm

The core design philosophy of the rounding off algorithm is to pre-decompose the

target vector through the inverse operation of the trapdoor basis, and after integer rounding over the ring, map it back to the discrete space spanned by the lattice basis, ultimately solving for the error term corresponding to the lattice vector that is closest in Euclidean distance to the target vector.

For the target vector to be decoded in the NTRU cryptosystem, $c = hs + e \in (\mathcal{R} \bmod q)$ (where s is the message mapping vector and e is the error vector sampled from a discrete Gaussian distribution), it satisfies $\begin{pmatrix} c \\ 0 \end{pmatrix} \in \mathcal{L}_{h,q} + \begin{pmatrix} e \\ -s \end{pmatrix}$.

The RO algorithm achieves an approximate recovery of the closest lattice vector by performing an inverse mapping of the target vector using the trapdoor basis and executing coefficient-level rounding. Its computational form is

$$\begin{pmatrix} e' \\ -s' \end{pmatrix} = \begin{pmatrix} c \\ 0 \end{pmatrix} - B_{f,g} \left\lfloor B_{f,g}^{-1} \begin{pmatrix} c \\ 0 \end{pmatrix} \right\rfloor$$

Here, $\lfloor \cdot \rfloor$ denotes the coefficient-wise round-to-nearest integer operator over the polynomial ring. Utilizing the NTRU trapdoor structure, its inverse matrix can be expressed as $B_{f,g}^{-1} = \frac{1}{q} \begin{pmatrix} F & -G \\ -f & g \end{pmatrix}$.

When $\begin{pmatrix} F & -G \\ -f & g \end{pmatrix} \begin{pmatrix} e \\ -s \end{pmatrix} = \begin{pmatrix} Fc \bmod q \\ -fc \bmod q \end{pmatrix}$ holds true, the algorithm can achieve error-free decoding. The sufficient condition for the RO algorithm to decode correctly is

$$\max\{\|fe + gs\|_\infty, \|Fe + Gs\|_\infty\} \leq \frac{q}{2}$$

Here, $\|\cdot\|_\infty$ represents the infinity norm of a polynomial vector, which is the maximum absolute value among all coefficients of the polynomial.

The RO algorithm is highly efficient and requires only integer operations. However, its decoding distance is small, constrained by the large norms of (G, F) in the trapdoor basis, there exists an $O(\sqrt{n})$ gap in the decoding range.

3.1.2 Nearest Plane (NP) Algorithm

The core design philosophy of the Nearest Plane algorithm is to preprocess the trapdoor basis through Gram-Schmidt (GS) orthogonalization to obtain a sequence of mutually orthogonal basis vectors, and then, through successive projection and rounding operations, map the target vector layer by layer onto the nearest plane spanned by the orthogonal basis. This ultimately accomplishes the solving of the closest lattice

vector, thereby tightening the norm constraint bound for decoding.

For the NTRU trapdoor basis $B_{f,g}$, Gram-Schmidt orthogonalization is first applied to it to obtain the orthogonalized basis matrix $B_{f,g}^* = \begin{pmatrix} g & G^* = G - vg \\ f & F^* = F - vf \end{pmatrix}$, where the orthogonalization coefficient is $v = \frac{\overline{f\bar{f} + G\bar{g}}}{f\bar{f} + g\bar{g}}$, and $\bar{\cdot}$ denotes the conjugate operation over the polynomial ring.

The NP algorithm achieves layered iterative decoding based on the orthogonalized basis. The sufficient condition for its correct decoding is

$$\max\{\|fe + gs\|_\infty, \|F^*e + G^*s\|_\infty\} \leq \frac{q}{2}$$

The NP algorithm has a large decoding distance, but it relies on high-precision Gram-Schmidt orthogonalization and floating-point operations, making it unsuitable for direct application in lightweight implementations.

3.1.3 GGH-Micciancio Paradigm

The core of GGH lattice encryption is to use the trapdoor basis as the private key and recover the message by decoding the closest lattice point using the nearest plane algorithm, without requiring additional mask parameters. Adapting this paradigm to the NTRU lattice, retain the NTRU polynomial ring $R = \mathbb{Z}[x]/(x^n + 1)$ (where n is a power of 2) and the public key structure $h = f^{-1}g \bmod q$; the private key is no longer the short vector (f, g) , but the NTRU trapdoor basis $B_{f,g} = \begin{pmatrix} g & G \\ f & F \end{pmatrix} \in R^{2 \times 2}$, which satisfies the NTRU equation $gF - fG = q$. The encryption paradigm closely follows GGH, with the ciphertext constructed as $c = hm + e \bmod q$ (where e is a small error term). The absence of the mask modulus p fundamentally eliminates the parameter limitations of traditional NTRU.

3.1.4 NTRU Decoder Fusing RO and NP Algorithms

The key to achieving maskless decryption without p is using a deterministic NTRU decoder that fuses the Babai Rounding Off (RO) and Nearest Plane (NP) algorithms solves the issues of the GGH nearest plane algorithm relying on floating-point operations and the limited decoding distance of traditional NTRU. The core of this decoder is to replace the floating-point Gram-Schmidt orthogonalization with a

shift-number approximation, introduce parameters to adapt to the distribution differences between messages and errors, and front-load complex computations to the key generation stage, ultimately achieving the efficiency of RO combined with the decoding distance of NP.

(1) Core Idea

Replace the high-precision Gram-Schmidt basis (G', F') of the NP algorithm with a integer approximated orthogonalized basis (G^*, F^*) , where $(G', F') = (G - g[v]_{q'}, F - f[v]_{q'}), v = \frac{\gamma^2 F \bar{f} + G \bar{g}}{\gamma^2 f \bar{f} + g \bar{g}}, [v]_{q'} = [v q'] / q', \gamma = \sigma_e / \sigma_s$ is the standard deviation ratio between the error e and the message s , and q' is the shift-number approximation precision parameter. This retains the large decoding distance of the NP algorithm while avoiding floating-point operations.

(2) Main Steps

Precompute the auxiliary polynomial w : In the key generation stage, calculate $w = q'[v]_{q'}$ using the ComputeVec algorithm. As part of the private key, this front-loads the complex fractional and rounding computations, allowing direct invocation during the decryption stage.

Deterministic integer decoding: In the decryption stage, the Decode algorithm utilizes the trapdoor basis $B_{f,g}$ and the precomputed w , to construct two linear equations to solve for the two unknowns (e, s) , i.e., $\begin{pmatrix} F & -G \\ -f & g \end{pmatrix} \begin{pmatrix} e \\ -s \end{pmatrix} = \begin{pmatrix} Fc \bmod q \\ -fc \bmod q \end{pmatrix}$

(3) Core Condition for Decoding Correctness

Compared to $\|pgr + fm\|_\infty < q/2$ of traditional NTRU and $\|e_0g - e_1f + e_2\|_\infty < q/4$ of Ring-LWE, the decoding condition is more relaxed, $\|gm + fe\|_\infty < q/2$, this allows a smaller modulus q .

3.2 Sampling Algorithms

To generate an NTRU private key pair (f, g) that satisfies the trapdoor quality constraints, this scheme adopts a sampling algorithm based on the Fourier domain. Centered around the trapdoor quality constraint, this algorithm directly constructs a sampling distribution satisfying the norm bounds within the embedding space, ensuring

that the generated keys meet the predefined interval limits under all embeddings. Meanwhile, compared to the traditional "sample-verify-resample" method based on the coefficient domain, this algorithm avoids the uncertain computational overhead caused by repetitive sampling, thereby reducing overall complexity. In terms of distribution design, by constructing an approximately uniform sampling strategy within the embedding domain, it guarantees the statistical unbiasedness of the generated key pair and avoids potential distribution bias issues. Furthermore, the algorithm introduces the parameter α as a trapdoor quality control factor, allowing for an adjustable trade-off between the sampling range and decoding performance. Based on the above design, the algorithm directly generates continuous values satisfying the constraints in the Fourier embedding domain. Combined with a rounding error compensation mechanism, it stably maps them to integer elements on the cyclotomic ring, thus completing the construction of the private key pair.

Under the hybrid sampler, the algebraic quality constraint for the NTRU lattice trapdoor requires that for all complex embeddings φ_i of the cyclotomic ring $\mathcal{R}$, the ring element pair (f, g) must satisfy $\frac{q}{\alpha^2} \leq |\varphi_i(f)|^2 + |\varphi_i(g)|^2 \leq \alpha^2 q$ (where α is the trapdoor quality and q is the modulus). This algebraic constraint can be equivalently transformed into a geometric constraint in the Fourier domain (embedding domain), for each embedding φ_i , the magnitude pair $(|\varphi_i(f)|, |\varphi_i(g)|)$ must fall within the annular region in the first quadrant of the plane, denoted as $A_\alpha^+ = A^+(\sqrt{q/\alpha}, \alpha\sqrt{q})$ ($A^+(r, R)$ represents the upper-right quadrant annulus with an inner radius r and outer radius R).

A bijective relationship exists between the coefficient domain and the Fourier domain (embedding domain) of the cyclotomic ring, and the embeddings satisfy conjugate symmetry ($\varphi_{d/2+i}(f) = \overline{\varphi_i(f)}$). Utilizing this property, one can circumvent the drawbacks of indirect sampling in the coefficient domain and directly sample the magnitudes of $(\varphi_i(f), \varphi_i(g))$ in the Fourier domain, making the sampling results naturally fit the geometric constraints of the trapdoor quality. Simultaneously, by leveraging conjugate symmetry, independent sampling is only required for $d/2$ embeddings, while the remaining embeddings are completed via complex conjugation. This halves the sampling volume while ensuring sampling completeness, significantly

reducing computational complexity.

The sampling in the Fourier domain yields continuous complex values. After transforming them back to the coefficient domain via the inverse Fourier transform, they must be converted into integer ring elements of the cyclotomic ring through coefficient-wise nearest integer rounding. However, rounding introduces errors uniformly distributed in $[-1/2, 1/2)$, which can cause the embedded values to deviate from the target annulus. To offset this deviation, the algorithm does not sample directly in the target annulus A_α^+ . Instead, it constructs a shrunken intermediate annulus $A^+(r, R)$ (where $r > \sqrt{q/\alpha}$ and $R < \alpha\sqrt{q}$) as the actual sampling domain. By appropriately shrinking the sampling range, it ensures that the embedded values after rounding still satisfy the target quality constraints.

To avoid a degradation in attack resistance caused by key distribution bias, the algorithm employs a uniform sampling strategy in polar coordinates within the annular region: it samples uniformly on the square of the polar radius rather than directly on the polar radius, while simultaneously performing uniform random sampling on the polar angle and the phase of the complex embeddings. This ensures that the magnitude pairs $(|\varphi_i(f)|, |\varphi_i(g)|)$ are uniformly distributed within the annulus, making the generated (f, g) statistically unbiased and enhancing the key's attack resistance from the perspective of the sampling distribution.

3.3 Optimization of NTRU Equation Solving

Solving the NTRU equation for the short polynomial pair (F, G) is the core computational bottleneck in the key generation stage. Its efficiency, resource consumption, and stability directly impact the engineering implementation capability of the scheme. Combined with the design requirements of the parameter system in this paper, targeted optimization designs are carried out to address the pain points existing in traditional NTRU equation solving, such as the large overhead of recursive stack space, the low efficiency of iterative reduction, and numerous invalid computations.

3.3.1 Iterative Reconstruction and Stack Space Control Design

Traditional NTRU equation solving is often implemented recursively with a recursion depth of $\log_2 n$. An iterative reconstruction approach is adopted, referencing the core logic of TowerSolverI, to transform the recursive structure into an iterative

execution while preserving the solving precision of the original algorithm. The core logic is that in each iteration layer, if the short polynomial pair (f, g) cannot be stored in the temporary buffer, (f, g) is recomputed through deterministic regeneration logic, thereby avoiding the accumulation of stack space caused by recursive calls. In terms of implementation, a loop structure is used to simulate the recursion levels, and intermediate results are stored in a fixed-size array buffer, rather than relying on stack frames to store recursive parameters.

3.3.2 Adaptation and Tuning of the ReducePartial Iteration Rule

The traditional Reduce algorithm relies on f, g, F, G simultaneously to compute the reduction coefficient $k = \left\lfloor \frac{Ff^* + Gg^*}{ff^* + gg^*} \right\rfloor$, which features high computational complexity and requires storing the complete G polynomial, rendering it unsuitable for deployment in resource-constrained scenarios. We introduce the lightweight ReducePartial variant and conduct adaptation and tuning targeting its iterative properties and the modulus q characteristics of the parameter set in this paper. The core improvement is that ReducePartial calculates an approximate value $k \approx \left\lfloor \frac{F}{f} \right\rfloor$ relying only on f and F , eliminating the involvement of G and g . To tackle the problems of reduced reduction efficiency and elevated failure rate caused by approximate computation in ReducePartial, we adjust the `reduce_bits` parameter of the `ntru_profile` structure to decrease the number of assumed bit reductions per iterate on.

3.3.3 Lightweight Intermediate Verification Mechanism

In the conventional solving process, the validity of intermediate iterations is verified by checking whether the identity $fG - gF = q$ holds modulo a small prime p . However, ReducePartial does not retain G and g , so it cannot adopt this verification scheme. This causes invalid iterations to persist until the solving process completes, wasting computational resources.

We design a lightweight intermediate verification rule adapted to ReducePartial to enable early termination of invalid iterations. The verification logic is as follows: when the coefficient length of F meets the condition that the top 31-bit segment can be

removed, we check whether the sign bit of this top segment is redundant with that of the next segment. If the iteration deviates from expectations, the top segment will not satisfy the sign redundancy property, and subsequent reduction steps will fail to effectively reduce the coefficient length. This verification only requires checking the sign consistency of integer bit segments, with no additional overhead from modular operations.

3.4 Parameter Selection

The specific requirements for parameter selection are that the polynomial ring is a cyclotomic ring, the security strength targets of 128-bit, 256-bit, and 512-bit are satisfied, and the parameters n, b, k, q, σ_f, q' are determined such that the decryption failure probability is sufficiently small.

First, for the cyclotomic ring modulo the polynomial $x^n + 1$, when $n = 2^k$ and the prime $q \equiv 1 \pmod{2n}$, the polynomial $x^n + 1$ aligns with the $2n$ -th cyclotomic polynomial, and a full Number Theoretic Transform (NTT) can be performed. To reduce bandwidth consumption, we use a partial NTT in our implementation, so the prime q satisfies $q \equiv 1 \pmod{\frac{2n}{k}}$ ($k \in \mathbb{Z}$).

After fixing n and q , we adjust the remaining parameters to meet the required security strength. The security strength is evaluated by converting the candidate parameters into those for the NTRU problem and the LWE problem separately, then assessing the security strength via a complexity estimation function (the security strength is formulated as $c^2(d - \beta)2^{0.292*\beta}$, the expression for quantum security strength is $c^2(d - \beta)2^{0.257*\beta}$).

```
try:
    temp1 = NTRU.primal_usvp(params, red_shape_model="gsa")
    print("primal")
    print(temp1)
    print("-----")
except Exception:
    pass

try:
    temp3 = NTRU.primal_hybrid(params, mitm=True, babai=False)
    print("primal hybrid")
    print(temp3)
    print("-----")
except Exception:
    pass
```

Figure 1 Partial code for security strength calculation

Then compute the decryption failure probability of the encryption algorithm. The procedure for calculating the decryption failure probability is as follows.

s. Compute sequentially $\gamma = \frac{\sigma_e}{\sigma_s}, I(\gamma, r, R) = \frac{\ln\left(\frac{R^2}{r^2}\right)}{\gamma(R^2 - r^2)}, r = \left(\frac{1}{3}\alpha + \frac{2}{3\alpha}\right)\sqrt{q}, R = \left(\frac{2}{3}\alpha + \frac{1}{3\alpha}\right)\sqrt{q}$, $\sigma_2 = \sigma_s \left(\frac{q\gamma}{\sqrt{n}\sigma_f} \sqrt{I(\gamma)} + \frac{n^{1.5}\sqrt{\gamma^2+1}}{2q'} \sigma_f \right), \sigma_1 = \sqrt{n}\sigma_f\sigma_s\sqrt{\gamma^2+1}, \tau = \min\left\{\frac{q}{2\sqrt{2}\sigma_1}, \frac{q}{2\sqrt{2}\sigma_2}\right\}$, Finally, the decryption failure probability is obtained as $P_{fail} = 2n \cdot (1 - \text{erf}(\tau))$.

```
sigma_s = mp.mpf('0.5')
sigma_e = mp.sqrt((k**2 - 1) / 12)
gamma = sigma_e / sigma_s

sqrt_q = mp.sqrt(q)
r = ((mp.mpf('1') / 3) * alpha + (mp.mpf('2') / 3) * (1 / alpha)) * sqrt_q
R = ((mp.mpf('2') / 3) * alpha + (mp.mpf('1') / 3) * (1 / alpha)) * sqrt_q

I = I_gamma(gamma, r, R)

sigma_1 = mp.sqrt(n) * sigma_f * sigma_s * mp.sqrt(gamma**2 + 1)

sigma_2 = sigma_s * (
    (mp.sqrt(q * gamma) / (mp.sqrt(n) * sigma_f)) * mp.sqrt(I)
    + (n**mp.mpf('1.5') * mp.sqrt(gamma**2 + 1)) / (2 * q0 * sigma_f)
)

tau_1 = q / (2 * mp.sqrt(2) * sigma_1)
tau_2 = q / (2 * mp.sqrt(2) * sigma_2)
tau = min(tau_1, tau_2)

failure_prob = 2 * n * mp.erfc(tau)
failure_log2 = mp.log(failure_prob, 2)
```

Figure 2 Partial code for decryption failure probability calculation

The final selected parameters are presented in **Table 1**.

The finally selected parameters, τ , and decryption failure probability are shown in **Table 1**.

Table 1 Parameter Set Selection and Decryption Failure Probability

Parameter Set	n	(b, k, q)	σ_f	α	q'	τ	Decryption Failure Probability
CTL_128	512	(128, 2, 257)	0.48	1.17	64513	11.83	2^{-196}
CTL_256	1024	(192, 4, 769)	0.47	1.23	64513	14.76	2^{-308}
CTL_512	2048	(416, 8, 3329)	0.64	1.40	64513	18.88	$2^{-426.1}$

The security strengths corresponding to these three groups of parameters are

presented in **Table 2**.

Table 2 Security Strength Corresponding to Different Parameter Sets

Parameter Set	Classical Security Strength		Quantum Security Strength	
	Recovery Key	Forgery Signature	Recovery Key	Forgery Signature
CTL_128	$2^{137.6}$	$2^{131.2}$	$2^{122.3}$	$2^{123.9}$
CTL_256	$2^{256.3}$	$2^{270.8}$	$2^{227.2}$	$2^{254.2}$
CTL_512	$2^{512.5}$	$2^{534.8}$	$2^{452.5}$	$2^{488.6}$

3.5 Principles of Engineering Implementation for Encoding, Storage and Decoding

The engineering implementation of encoding, storage and decoding serves as a core link in the practical deployment of cryptographic systems. The design of encoding and storage is centered on three core data objects: public key, private key, and ciphertext. With the prerequisite of ensuring correct data parsing and full compatibility with encryption/decapsulation algorithms, the core design objective is to compress storage overhead through coefficient grouping, high-low bit splitting, customized base encoding and other strategies, while reducing the computational complexity of encoding and decoding.

Relying on the NTRU trapdoor basis and precomputed auxiliary polynomials, the decoding process adopts engineered simplification strategies to efficiently recover message information from ciphertext-related polynomials and avoid critical difficulties in engineering implementation. A unified header identification specification is adopted for the encoding format of all data objects, with refined adaptation tailored to the modulus characteristics of different parameter sets.

3.5.1 Unified Design Specification for Encoding and Storage

(1) **Prefix header byte:** All encoded objects start with a single-byte header serving as a format identifier. It uniquely labels the object type (public key/private key/ciphertext) and the corresponding parameter set to prevent format mixing between

different objects or security levels;

(2) **Integer encoding principle:** All polynomial coefficients are represented in two's complement of signed integers. The entire encoding process relies on integer arithmetic with no floating-point operations;

(3) **Grouped compression rule:** A fixed-length grouping compression scheme is applied to polynomial coefficients. Redundant high-bit information is merged via radix encoding to maximize the compression ratio under the constraint of lossless compression.

3.5.2 Public Key Encoding Specification

The public key is an element $h = f^{-1}g \bmod q$ over the polynomial ring $R_q = \mathbb{Z}[x]/(x^n + 1)$, where n is the ring dimension, and q is the modulus. Each coefficient of the public-key polynomial is an integer modulo q , with a range of $[-\lceil q/2 \rceil + 1, q - \lceil q/2 \rceil]$.

(1) Public Key Encoding Specification for CTL-128 Parameter Set ($n = 512, q = 257$)

Coefficient range: 0~256 (integer representation modulo 257); each coefficient carries 9 valid bits.

Grouping rule: Group 8 consecutive coefficients into one block, with a total size of 65 bits per block.

Encoding logic: Each coefficient is split into a lower 4-bit part and an upper half part (range 0~16). The upper half parts of 8 coefficients are combined and encoded in base-17 into 33 bits ($17^8 = 6975757441 < 2^{33} = 8589934592$). The lower 4-bit parts of 8 coefficients are concatenated into 32 bits. The two parts are joined to complete the block encoding.

(2) Encoding Specification for CTL-256 Parameter Set ($n = 1024, q = 769$)

Coefficient range: 0~768 (integer representation modulo 769); each coefficient carries 10 valid bits.

Grouping rule: Group 5 consecutive coefficients into one block, with a total size of 48 bits per block.

Encoding logic: Each coefficient is split into a lower 2-bit part and an upper half part (range 0~192). The upper half parts of 5 coefficients are combined and encoded

in base-193 into 38 bits ($193^5 = 267785184193 < 2^{38} = 274877906944$). The lower 2-bit parts of 5 coefficients are concatenated into 10 bits. The two parts are joined to complete the block encoding, adapted to the coefficient range of modulus 769 to achieve lossless compression.

(3) Encoding Specification for CTL-512 Parameter Set ($n = 2048, q = 3329$)

Coefficient range: $0 \sim 3328$ (integer representation modulo 3329); each coefficient carries 12 valid bits.

Grouping rule: Group 4 consecutive coefficients into one block, with a total size of 47 bits per block.

Encoding logic: Each coefficient is split into a lower 6-bit part and an upper half part (range $0 \sim 52$). The upper half parts of 4 coefficients are combined and encoded in base-53 into 23 bits ($53^4 = 7892481 < 2^{23} = 8388608$). The lower 6-bit parts of 4 coefficients are concatenated into 24 bits. The two parts are joined to complete the block encoding, adapted to the coefficient range of modulus 3329 to achieve lossless compression.

3.5.3 Ciphertext Encoding Specification

A ciphertext consists of two parts: the core polynomial c_1 and the fixed-length binary hash payload c_2 . Here, c_1 is a polynomial with small-range signed integer coefficients and serves as the primary target for ciphertext encoding and compression; c_2 is fixed-length binary data related to message hashing with no compressible space. The core of ciphertext encoding is to compress and encode c_1 , then append c_2 directly to the end of the encoded byte stream of c_1 to form the complete ciphertext.

(1) Compression Encoding Rules for Polynomial c_1

CTL-128 Parameter Set ($n = 512, q = 257$)

Coefficient range of c_1 : $-64 \sim +64$; each coefficient contains 7 valid bits.

Grouping rule: 8 consecutive coefficients form one block, occupying 57 bits per block. A radix merging encoding scheme consistent with that of the public key is adopted, adapted to the smaller coefficient range.

CTL-256 Parameter Set ($n = 1024, q = 769$)

Coefficient range of c_1 : $-96 \sim +96$; each coefficient contains 8 valid bits.

Grouping rule: 5 consecutive coefficients form one block, occupying 38 bits per

block. High-order information is merged and encoded in base 193.

CTL-512 Parameter Set ($n = 2048, q = 3329$)

Coefficient range of c_1 : $-208 \sim +208$; each coefficient contains 9 valid bits.

Grouping rule: 4 consecutive coefficients form one block, occupying 35 bits per block. A radix merging encoding scheme consistent with that of the public key is adopted, adapted to the smaller coefficient range, and high-order information is merged and encoded in base 53.

(2) Concatenation Specification for Binary Payload c_2

c_2 is a fixed-length binary string output by a hash function, with no additional compression encoding. It is directly concatenated in big-endian byte order to the end of the encoded byte stream of c_1 .

The length of c_2 corresponds to the security level: 16 bytes (128-bit security) for CTL-128, 32 bytes (256-bit security) for CTL-256, and 64 bytes (512-bit security) for CTL-512.

The complete concatenated ciphertext has a fixed length with no variable-length fields or padding bytes, ensuring compact ciphertext size.

3.5.4 Private Key Encoding Specification

The private key is core sensitive information. Two encoding schemes are designed: a long format (full storage) and a short format (lightweight storage), which are suitable for high-frequency encryption/decryption scenarios and storage-constrained scenarios respectively. The two schemes provide equivalent cryptographic security and differ only in storage overhead and decoding overhead.

The core components of the private key include the NTRU trapdoor basis $B_{f,g} = \begin{pmatrix} g & G \\ f & F \end{pmatrix}$, the decoding auxiliary polynomial w , the public key h , the key generation seed, and the decryption backup parameter r

(1) General Bit Compression Rules for Core Polynomials in the Private Key

Fixed bit compression rules are defined for polynomials f, g, F, G, w in the private key according to their actual coefficient ranges. All rules are implemented using integer two's complement or direct bit storage with no extra encoding overhead. The specific rules are:

Coefficients of polynomials f and g are encoded and stored in 4-bit two's

complement;

Coefficients of polynomials F and G are encoded and stored in 6-bit format;

Coefficients of the auxiliary polynomial w are encoded and stored in 17-bit format;

The public key h , as a sub-object of the private key, uses exactly the same public key encoding format as in Section 3.5.2 to avoid repeated encoding during decryption.

(2) Long-Format Private Key Encoding Specification

The long-format private key is a full storage mode that stores all parameters and polynomials required for key generation and decryption. No regeneration computation is needed during decoding; the data can be read directly for decryption operations, making it suitable for high-frequency key usage scenarios. The long format is encoded and stored in a fixed data order. All sub-objects are compressed using the above general rules and stored directly. The complete data stream (in storage order) is:

32-byte random seed (for generating polynomials f, g) $\rightarrow$ 32-byte backup parameter r (for fault tolerance in case of decryption failure in the FO transform) $\rightarrow$ compressed polynomial f $\rightarrow$ compressed polynomial g $\rightarrow$ compressed polynomial F $\rightarrow$ compressed polynomial G $\rightarrow$ compressed polynomial w $\rightarrow$ public key h encoded under public key rules.

(3) Short-Format Private Key Encoding Specification

The short-format private key stores only core non-regenerable sensitive information. All other polynomials and parameters are regenerated via a deterministic pseudorandom process. It minimizes storage usage under acceptable decoding overhead and is suitable for storage-resource-constrained scenarios.

Core stored content: Only the 32-byte key generation random seed and polynomial F are stored, with no other redundant fields. The regeneration logic for the remaining sub-objects is.

Value r and polynomials f, g : Regenerated from the 32-byte seed using the same deterministic pseudorandom algorithm as in key generation, without re-sampling or re-selection;

Polynomial G : Computed directly modulo q from the core NTRU equation $gF - fG = q$, without re-solving the NTRU equation (the most time-consuming step in key generation);

Auxiliary polynomial w : Re-computed by the ComputeVec algorithm from the regenerated f, g, F, G ;

Public key h : Re-computed according to $h = f^{-1}g \bmod q$.

3.5.5 Quantification of Storage Requirements

Based on the above encoding and storage principles, we present the actual storage overhead (unit: byte) including a 1-byte header identifier for the three parameter sets: CTL-128 (128-bit security), CTL-256 (256-bit security), and CTL-512 (512-bit security). The ciphertext refers to the practical deployment form with the Fujisaki-Okamoto (FO) transform, which is the core storage format in KEM engineering applications. The detailed quantitative data are shown in Table 3, The bandwidth for direct encoded storage of each parameter set is shown in Table 4.

Table 3 Storage Requirement Statistics for Each Parameter Set

Parameter Set	Public Key(bytes)	Ciphertext (bytes)	Private Key (Short Format)(bytes)	Private Key (Long Format)(bytes)
CTL-128	521	473	417	2953
CTL-256	1230	1006	801	6030
CTL-512	3009	2305	3105	15617

Table 4 Bandwidth of Direct Storage After Encoding for Each Parameter Set

Parameter Set	Dimension n	Public Key (bytes)	Ciphertext (bytes)
CTL-128	512	576	480
CTL-256	1024	1280	1056
CTL-512	2048	3072	2368

3.5.6 Decoding Principle

Based on the NTRU trapdoor basis and precomputed auxiliary polynomials, the polynomial parameters e' and s' containing message information are recovered from the ciphertext-related polynomials. All operations are performed on integer-coefficient

polynomials and require polynomial computations over finite fields modulo q , q' , and Q .

In engineering implementation, targeted simplifications are made for the mathematical objective of decoding, significantly reducing computational complexity. Mathematically, decoding requires simultaneous recovery of the polynomials e' and s' . However, in the actual decapsulation process, the validity of decoding results can be verified via an encapsulation verification mechanism. Thus, decoding can be completed by recovering only the core parameter s' without additional computation of e' , directly cutting the core computational load by half. Meanwhile, each coefficient of the polynomial s' can take only two values: $1/2$ and $-1/2$. The core task of decoding is further simplified to distinguishing these two binary values rather than complex numerical solving, greatly streamlining the operational logic.

To completely avoid floating-point operations and maintain consistency in the integer arithmetic framework, a dedicated integer arithmetic conversion technique is designed to address the fractional coefficients of s' . Instead of directly recovering s' in fractional form, the integer-valued $q'Qs'$ is computed, converting all fractional operations into pure integer arithmetic. This fundamentally eliminates precision loss and hardware implementation complexity caused by floating-point operations. Furthermore, to improve the stability and effectiveness of the final solving step, a small prime distinct from q and q' is introduced as an extra modulus in this stage. Binary distinction of the coefficients of s' is achieved via modular arithmetic, effectively mitigating numerical overflow in integer operations.

NTRU Decoding Framework: Bilinear equations are constructed using the NTRU trapdoor basis to recover the encrypted message and error simultaneously, freeing the scheme from dependence on the masking modulus p . This enables smaller moduli, more compact parameters, and higher security strength.

Ciphertext Construction: $c = hs + e \bmod q$, where h is the NTRU public key, s is the binary message polynomial, and e is the small-norm error polynomial.

Bilinear Equation Construction: Based on the NTRU trapdoor basis $B_{f,g} = \begin{pmatrix} g & G \\ f & F \end{pmatrix}$ (satisfying $gF - fG = q$), two independent linear equations can be constructed: $fc \bmod q = gm + fe$, $Fc \bmod q = Gm + Fe$.

Nature of Decryption: The two unknowns s and e are solved using the above linear equations, which is equivalent to solving the Closest Vector Problem (CVP) on the NTRU lattice. Without relying on the masking modulus p , the decapsulation boundary constraints are significantly relaxed.

3.6 Description of Differences

This algorithmic proposal is built upon the BAT framework in its underlying cryptographic architecture. While drawing on existing advanced theoretical frameworks, this scheme fully respects and avoids infringing upon the legitimate rights and interests of others. It introduces substantial original designs and profound modifications to address the engineering pain points of existing schemes, such as low efficiency in trapdoor generation, insufficient stability in equation solving, and incomplete coverage of security levels. The intellectual property and core innovations of this scheme are reflected in the design of the hybrid sampler, the lightweight reconstruction of the iterative solving of the NTRU equation, and the independent derivation of the compact parameter system.

(1) High-quality Generation of (f, g) via Hybrid Sampler

This scheme introduces a hybrid sampler for NTRU lattices, directly constraining the trapdoor quality from the perspective of distribution design. By utilizing the Fourier embedding structure of cyclotomic rings, it performs approximate uniform sampling in the frequency domain. Combined with a rounding error compensation mechanism, this allows the sampling results to naturally satisfy the geometric constraints of the trapdoor basis. This method effectively offsets the errors caused by coefficient rounding, improves the sampling success rate, reduces the fluctuation in trapdoor quality, and enhances the algorithm's stability.

(2) Optimization of Judgment Conditions and Processes for NTRU Equation Solving

Addressing the main computational bottleneck in the key generation phase, this scheme adds preliminary validity judgment conditions before solving the NTRU

equation and formulates an adaptive iteration termination strategy to reduce meaningless solving loops. In the underlying implementation, it transforms the traditional recursive structure into iterative execution, introducing the ReducePartial iterative optimization strategy and its accompanying lightweight intermediate verification mechanism. This reduces the overall failure rate and the waste of computational resources, significantly improving the overall efficiency and operational stability of the key generation phase.

(3) Multi-Security-Level Parameter System and Compact Engineering Implementation

Existing frameworks typically lack a fully covered security parameter system. This scheme expands the high-security-level parameter sets, constructing a comprehensive parameter set system covering multiple security gradients of 128-bit, 256-bit, and 512-bit (CTL-128, CTL-256, CTL-512). Through parameter tuning, it adopts a smaller computational modulus under the same security level, which drastically reduces the complexity of algorithm implementation and achieves extremely small public key and ciphertext sizes. This scheme reaches an optimal balance among algorithm security, operational efficiency, and decryption failure rate, demonstrating strong universal applicability across different scenarios.

4. Algorithm Description

This scheme inherits the advantage of compact parameters from the NTRU-based cryptosystem and carries out targeted optimizations on core modules including trapdoor generation, auxiliary vector computation, and lattice decoding. A novel Fourier-domain cyclic sampling algorithm is designed to generate short polynomial pairs (f, g) with high trapdoor quality. By introducing an error compensation factor ξ and scaled cyclic parameters, the NTRU decoding procedure is improved, which effectively offsets errors caused by coefficient rounding and further reduces the decryption failure probability.

The encryption process is built upon the hardness assumption of Ring Learning

with Rounding (Ring-LWR), which ensures computational security while achieving ciphertext size compression. The decryption process recovers the message and error term by solving linear equations with the NTRU trapdoor basis, abandoning high-precision floating-point arithmetic and realizing fully integer-based operations. The proposed scheme features compact public/private keys, efficient encryption/decryption, and low decryption failure probability. It satisfies cryptographic requirements for lightweight devices and low-bandwidth communication scenarios, providing a new implementation framework for the engineering deployment of post-quantum cryptosystems.

This scheme is designed based on the NTRU lattice cryptosystem, focusing on four key components: efficient trapdoor generation, compact ciphertext construction, deterministic integer decoding, and highly robust key encapsulation. The overall execution flow follows. Sampling to generate short polynomial pairs $\rightarrow$ Construct NTRU trapdoor basis $\rightarrow$ Generate public/private keys $\rightarrow$ Ring-LWR-based encryption $\rightarrow$ Trapdoor-assisted decryption $\rightarrow$ Key encapsulation/decapsulation. The basic encryption scheme achieves IND-CPA security, and the extended KEM scheme satisfies IND-CCA security.

This scheme adopts the Fourier-domain cyclic sampling algorithm *Sampler_fg*. Through scaled cyclic parameter design and embedded norm verification, it generates short polynomial pairs (f, g) that meet trapdoor quality requirements, effectively compensating errors from coefficient rounding and enhancing the effectiveness of the trapdoor basis. By judging the validity of the trapdoor basis in advance, the number of NTRU equation solvings is reduced, significantly improving performance in the key generation phase. Auxiliary vector computation and decoding procedures are optimized, and an error compensation factor ξ and distribution balancing parameter γ are introduced to achieve accurate recovery of messages and error terms, further lowering the decryption failure rate.

The underlying mathematical model of this scheme is the polynomial ring $R = \mathbb{Z}[x]/(x^n + 1)$, where $n = 2^l, l \in \mathbb{N}$. All operations are performed over this ring and its modulo- q residue class ring R/qR . Core parameters include the modulus q , block parameter k , trapdoor quality parameter α , error compensation factor ξ , and decryption failure rate control parameter q' . These parameters work together to

guarantee the security, compactness, and efficiency of the scheme.

The scheme consists of six core algorithms: KeyGen (key generation), Encrypt_base (basic encryption), Encrypt (encryption), Decrypt (decryption), Encapsulate (encapsulation), and Decapsulate (decapsulation). Targeted optimizations are made to address the inefficiency of traditional NTRU trapdoor generation, floating-point-dependent decoding, and redundant ciphertexts. The scheme overall implements IND-CPA-secure basic encryption and IND-CCA secure key encapsulation, suitable for lightweight post-quantum cryptography deployment.

4.1 Key Generation Algorithm

This algorithm forms the foundation of the entire cryptographic scheme. Its core outputs are the public key h and the private key, which contains the NTRU trapdoor basis $B_{f,g}$ and the decoding auxiliary vector w .

First, the algorithm adapts to the distribution characteristics of messages and errors using the distribution balancing parameter γ , $I(\gamma, r, R) = \frac{\ln\left(\frac{R^2}{r^2}\right)}{\gamma(R^2 - r^2)}$, where $r = \left(\frac{1}{3}\alpha + \frac{2}{3\alpha}\right)\sqrt{q}$, $R = \left(\frac{2}{3}\alpha + \frac{1}{3\alpha}\right)\sqrt{q}$. Then it invokes *Sampler_fg* to generate a short polynomial pair (f, g) that satisfies the trapdoor quality requirement. After passing dual checks for invertibility and trapdoor validity, the NTRU trapdoor basis is constructed and the decoding auxiliary vector w is computed. Traditional NTRU trapdoor generation adopts coefficient-domain sampling plus posterior filtering, which results in many invalid samples and large fluctuations in trapdoor quality. The proposed algorithm reduces invalid sampling through pre-positive validity judgment and uses high-quality (f, g) from Fourier-domain sampling to improve the efficiency and stability of key generation. Meanwhile, the auxiliary vector w is precomputed to advance complex operations from the decoding stage, avoiding high-precision floating-point operations during decryption.

Algorithm 1 Key Generation (KeyGen)

Input: Polynomial ring $\mathcal{R} = \mathbb{Z}[x]/(x^n + 1)$ ($n = 2^\ell, \ell \in \mathbb{N}$), $q \in \mathbb{N}, k \in \mathbb{N}, q' \in \mathbb{N}$, quality parameter α .

Output: Public key $h \in \mathcal{R}/q\mathcal{R}$, private key $(B_{f,g}, w)$

-
1. Compute the distribution balancing parameter: $\gamma = \sqrt{\frac{k^2-1}{3}}$.
 2. Invoke the *Sampler_fg* algorithm to obtain $(f, g) \in \mathcal{R}^2$.
 3. Invertibility check: if f or g is not invertible in $\mathcal{R}/q\mathcal{R}$, go back to Step 2.
 4. Trapdoor validity check: if $\left\| q\gamma \cdot \left(\frac{-\gamma\bar{f}}{\gamma^2 f\bar{f} + g\bar{g}}, \frac{\bar{g}}{\gamma^2 f\bar{f} + g\bar{g}} \right) \right\| > q\gamma \cdot \sqrt{I(\gamma, r, R)}$, then go back to Step 2.
 5. Generate public key: compute $h = f^{-1}g \bmod q$.
 6. Compute $(F, G) \in \mathcal{R}^2$ such that $gF - fG = q$;
 7. Construct the NTRU trapdoor basis: $B_{f,g} = \begin{pmatrix} g & G \\ f & F \end{pmatrix} \in \mathcal{R}^{2 \times 2}$;
 8. Compute auxiliary vector: $w = \text{ComputeVec}(B_{f,g}, q', \gamma)$;
 9. Output result: return $(h, (B_{f,g}, w))$.
-

4.2 Encryption Sub-algorithm

As a basic deterministic encryption step, it achieves ciphertext compression based on the Ring Learning with Rounding (Ring-LWR) assumption. Given a public key and a binary message s , it generates the intermediate ciphertext result c_1 by rounding $hs \bmod q$, computes the error term e , and screens valid results via norm checking simultaneously. The rounding operation of Ring-LWR is used to realize ciphertext compactness and reduce transmission and storage overhead. Meanwhile, invalid ciphertexts that are likely to cause decryption failures are eliminated in advance through error norm verification, thereby reducing the error rate of subsequent decryption.

Algorithm 2 Basic Encryption (Encrypt_base)

Input: Public key $h \in \mathcal{R}$, integers $q, k \in \mathbb{N}$, message $s \in \mathcal{M}' = (\mathcal{R} \bmod 2)$

Output: Ciphertext c_1 .

-
1. Compute $c_1 = \left\lfloor \frac{(hs \bmod q)}{k} \right\rfloor$.
 2. Compute the error term $e = (hs \bmod q) - kc_1$.
 3. Compute the distribution balancing parameter $\gamma \leftarrow \sqrt{\frac{k^2-1}{3}}$.
 4. If $||(\gamma s, e)|| > 1.08\sqrt{\frac{n(k^2-1)}{6}}$, then return failure symbol $\perp$;
 5. Output result: return intermediate ciphertext c_1 .
-

4.3 Encryption Algorithm

This algorithm implements IND-CPA secure randomized encryption based on Encrypt_base . Given a public key, a plaintext, and a random seed, it first generates a random binary ring element s . Then it invokes the basic encryption sub-algorithm to obtain c_1 . Finally, it generates c_2 using hash masking to form the final ciphertext pair (c_1, c_2) .

Deterministic encryption is vulnerable to statistical analysis attacks and cannot meet IND-CPA security requirements. By introducing a random seed to generate a pseudorandom s , this algorithm ensures that the same plaintext encrypted multiple times produces different ciphertexts. Meanwhile, hash masking is used to bind the plaintext with s , resisting replay and statistical analysis attacks, and achieving indistinguishability under chosen-plaintext attacks.

Algorithm 3 Encryption (Encrypt)

Input: Public key $h \in \mathcal{R}$, integers $q, k \in \mathbb{N}$, plaintext $m \in \mathcal{M}$, random seed

Output: Ciphertext (c_1, c_2)

1. Generate random binary ring element: $s \leftarrow \text{Sample}(U(\mathcal{R} \bmod 2); \text{seed})$.
 2. Invoke the basic encryption sub-algorithm: $\text{Encrypt}_{\text{base}}(h, q, k, s)$ to obtain c_1 . If $c_1 = \perp$, return $\perp$.
 3. Generate masked ciphertext: Compute $c_2 = \text{Hash}_m(s) \oplus m$, where Hash_m denotes a hash function with $\mathcal{M}$.
 4. Output result: return the ciphertext (c_1, c_2) .
-

4.4 Decryption Algorithm

This algorithm recovers the original plaintext from the ciphertext using the private key. It restores the message s through the decoding algorithm and obtains the plaintext by unmasking after validity verification.

Based on the trapdoor basis and auxiliary vector in the private key, the original plaintext is recovered from the ciphertext pair. First, c_1 is inverted by rounding to obtain c' . Then the *Decode* algorithm is invoked to recover the message s and the error term e . After dual verification of norm and ciphertext consistency, the plaintext is derived via hash unmasking.

This algorithm solves linear equations relying on the NTRU trapdoor basis, relaxing decryption boundaries. Meanwhile, fully integer decoding is realized using the precomputed auxiliary vector w , avoiding floating-point precision loss. The dual verification further filters malicious ciphertexts and erroneous decryption results, improving decryption reliability.

Algorithm 4 Decryption (Decrypt)

Input: Ciphertext (c_1, c_2) , public key $h \in \mathcal{R}$, private key $(B_{f,g}, w)$, integers $q, k, q' \in \mathbb{N}$

Output: Plaintext m

1. *Compute* $c' = c_1 k$.
2. Invoke the decoding algorithm: execute $\text{Decode}(B_{f,g}, w, q, q', 2, c')$ to obtain (e, s) .
3. Compute the distribution balancing parameter $\gamma = \sqrt{\frac{k^2 - 1}{3}}$.
4. Result validity verification: if $\|(ys, e)\| > 1.08 \sqrt{\frac{n(k^2 - 1)}{6}}$, then return failure symbol $\perp$.
5. Ciphertext consistency verification: if $c_1 = \left\lfloor \frac{(hs \bmod q)}{k} \right\rfloor$, unmask to reco

ver the plaintext: $m = \text{Hash}_m(s) \oplus c_2$, otherwise, return $\perp$.

6. Output result: return plaintext m .

4.5 Encapsulate

This algorithm realizes the binding generation of ciphertext and session key, which is the core step of Key Encapsulation Mechanism (KEM). Given a public key, it first randomly samples a plaintext m and invokes the encryption algorithm to generate a ciphertext pair. If encryption fails, the process restarts. Finally, the plaintext is mapped to a session key K via a hash function, and the ciphertext together with K is output.

The basic encryption scheme transmits plaintext directly, which is vulnerable to man-in-the-middle attacks in real communication and has a high risk of key reuse. This algorithm binds the one-time session key with the ciphertext to achieve “one-time one-key” and avoids the risk of long-term key leakage. The restart mechanism upon encryption failure ensures the reliability of encapsulation.

Algorithm 5 Encapsulation (Encapsulate)

Input: Public key $h \in \mathcal{R}$, integers $q, k \in \mathbb{N}$

Output: Ciphertext c , session key K

1. Sample a plaintext m from the uniform distribution over the plaintext space $\mathcal{M}$: $m \leftarrow \text{Sample}(U(\mathcal{M}))$.
2. Invoke the encryption algorithm $\text{Encrypt}(h, q, k, m, \text{Hash}_s(m))$ to generate the ciphertext pair (c_1, c_2) , where Hash_s is a hash function with range S .
3. If $(c_1, c_2) = \perp$, restart the entire encapsulation process.
4. Use the hash function Hash_k (with output space $\mathcal{K}$) to hash the plaintext m and generate the session key K : $K = \text{Hash}_k(m)$;
5. Return the ciphertext pair (c_1, c_2) and the session key K : $((c_1, c_2), K)$.

4.6 Decapsulate

As the inverse process of encapsulation, this algorithm achieves key recovery with

IND-CCA security. Given a ciphertext and a private key, it first invokes the decryption algorithm to obtain a candidate plaintext m' . It then verifies the consistency of the ciphertext through re-encryption checking. If the verification passes, m' is hashed into a session key K ; otherwise, it returns failure.

The basic decryption algorithm is vulnerable to chosen-ciphertext attacks (CCA), where an attacker can obtain key information by constructing malicious ciphertexts. This algorithm uses re-encryption checking to re-encrypt the candidate plaintext obtained from decryption and verifies whether it is consistent with the original ciphertext, effectively resisting attacks from malicious ciphertexts. The session key is generated only when the ciphertext is valid, ensuring the indistinguishability of the KEM under chosen-ciphertext attacks (IND-CCA).

Algorithm 6 Decapsulation (Decapsulate)

Input: Ciphertext (c_1, c_2) , public key $h \in \mathcal{R}$, private key $(B_{f,g}, w)$, integers $q, k, q' \in \mathbb{N}$

Output: session key K

1. Invoke the decryption algorithm $Decrypt((c_1, c_2), h, (B_{f,g}, w), q, k, q')$, to recover the candidate plaintext $m'm'$;
 2. If $m' \neq \perp$ and $(c_1, c_2) = Encrypt(h, q, k, m, Hash_s(m))$, return $K = Hash_k(m')$, otherwise, return the failure symbol $\perp$.
-

5. Basic Components

5.1 Sampling Algorithm `Sampler_fg`

Generate qualified short polynomial pairs that meet the trapdoor quality requirements $(f, g) \in \mathcal{R}^2$, providing the basis for the construction of the NTRU trapdoor basis.

Algorithm 7 Sampling Algorithm `Sampler_fg`

Input: polynomial ring $\mathcal{R} = \mathbb{Z}[x]/(x^n + 1)$ ($n = 2^\ell, \ell \in \mathbb{N}$), $q \in \mathbb{N}$ (modulus), target trapdoor quality $\alpha \in \mathbb{R}^+$, error compensation factor ξ (used for shrinking the annulus to offset rounding errors).

Output: qualified short polynomial pair $(f, g) \in \mathcal{R}^2$

1. **Compute the target annulus boundaries:** outer boundary $U = \alpha\sqrt{q}$,
inner boundary $L = \sqrt{q}/\alpha$;

2. **Compute the shrunk annulus parameters:**

Inner radius $r = \left(\frac{1-\xi}{3}\alpha + \frac{2+\xi}{3\alpha}\right)\sqrt{q}$, outer radius $R = \left(\frac{2+\xi}{3}\alpha + \frac{1-\xi}{3\alpha}\right)\sqrt{q}$ ($\xi = \frac{1}{3}$, $r = \left(\frac{1}{3}\alpha + \frac{2}{3\alpha}\right)\sqrt{q}$, $R = \left(\frac{2}{3}\alpha + \frac{1}{3\alpha}\right)\sqrt{q}$);

3. **Fourier-domain annulus sampling:**

For $i = 1$ to $n/2$ do:

a. Uniformly sample the squared magnitudes $x_i^2 \in [r^2, R^2]$, $y_i^2 \in [r^2, R^2]$,

and compute $x_i = \sqrt{x_i^2}$ (the embedding magnitude of f), $y_i = \sqrt{y_i^2}$ (the embedding magnitude of y);

b. Uniformly sample the complex phases $\theta_i \in [0, 2\pi)$, $\varphi_i \in [0, 2\pi)$ (to ensure isotropy of the embeddings in the complex plane);

c. Construct the embedding vectors $z_i = x_i \cdot e^{i\theta_i}$ (the i -th embedding corresponding to f), $w_i = y_i \cdot e^{i\varphi_i}$ (the i -th embedding corresponding to y);

4. **Inverse canonical embedding:** map the Fourier-domain vectors back to the polynomial domain, obtaining $\tilde{f} = \varphi^{-1}(z_i)$, $\tilde{g} = \varphi^{-1}(w_i)$;

5. **Coefficient rounding:** Round each coefficient $\tilde{f}$ and $\tilde{g}$ to the nearest integer, obtaining $f = \lfloor \tilde{f} \rfloor$, $g = \lfloor \tilde{g} \rfloor$;

6. **Embedding magnitude verification:** if there exists an embedding φ_i such that $|\varphi_i(f)|^2 + |\varphi_i(g)|^2 \notin [L^2, U^2]$, then re-execute Steps 3–6;

7. **Output result:** return (f, g) .

5.2 Auxiliary Vector Computation Algorithm ComputeVec

Compute the auxiliary vector w based on the NTRU trapdoor basis, providing support for the decoding algorithm and avoiding high-precision floating-point operations during decoding.

Algorithm 8 Auxiliary Vector Computation Algorithm ComputeVec

Input: trapdoor basis $B_{f,g} = \begin{pmatrix} g & G \\ f & F \end{pmatrix} \in \mathcal{R}^{2 \times 2}, q' \in \mathbb{N}, \gamma > 0;$

Output: $w \in \mathcal{R};$

1. Compute $v \leftarrow \frac{\gamma^2 F \bar{f} + G \bar{g}}{\gamma^2 f \bar{f} + g \bar{g}};$
 2. Return $w = q' \cdot \lfloor v \rfloor_{q'}.$
-

5.3 Decoding Algorithm Decode

Recover the message s and error term e from the ciphertext based on the NTRU trapdoor basis and the auxiliary vector, this is the core step of the decryption algorithm.

Algorithm 9 Decoding Algorithm Decode

Input: trapdoor basis $B_{f,g} = \begin{pmatrix} g & G \\ f & F \end{pmatrix} \in \mathcal{R}^{2 \times 2}, w \in \mathcal{R}$, positive integer q, q', Q ; ciphertext $c = (hs + e \bmod q) \in \mathcal{R}$, and $\mu_e, \mu_s \in \frac{1}{Q} \cdot \mathbb{Z};$

Output: polynomial pair $(e, s) \in \mathcal{R}^2;$

1. $(G_d, F_d) \leftarrow (q'G - gw, q'F - fw);$
 2. $c' \leftarrow (Qfc - f(Q\mu_e \mathbf{1}) - g(Q\mu_s \mathbf{1}) \bmod qQ);$
 3. $c'' \leftarrow (q'QFc - F(q'Q\mu_e \mathbf{1}) - G(q'Q\mu_s \mathbf{1}) - c'w \bmod qq'Q);$
 4. Solve the system of equations $\begin{pmatrix} f & g \\ F_d & G_d \end{pmatrix} \begin{pmatrix} e' \\ s' \end{pmatrix} = \frac{1}{Q} \cdot \begin{pmatrix} c' \\ c'' \end{pmatrix}$, to obtain $(e', s');$
 5. Return the result $e = e' + \mu_e \mathbf{1}, s = s' + \mu_s \mathbf{1}.$
-

6. Security Analysis

6.1 Security Assumption

NTRU assumption. Let $R_q^\times$ denote the set of invertible elements in R/qR . Let χ be a distribution over $R_q^\times$. The advantage of an adversary A in solving the decisional NTRU problem $\text{NTRU}_{R,q,\chi}$ is defined as

$$\text{Adv}_{R,q,\chi}^{\text{NTRU}}(A) = |\Pr[b = 1 | f, g \leftarrow \chi; b \leftarrow A(f^{-1}g \bmod q)] - \Pr[b = 1 | u \leftarrow U(R_q^\times); b \leftarrow A(u)]|$$

In our algorithm, $R = \mathbb{Z}[x]/(x^n + 1)$, and χ denotes the distribution of the secret keys f and g .

Regarding the hardness of the decisional NTRU problem over $R = \mathbb{Z}[x]/(x^n + 1)$, some studies already exist. In particular, as shown in [13], when χ is a discrete Gaussian distribution with standard deviation $\sigma = \tilde{\omega}(n\sqrt{q})$, the ratio of f and g is statistically indistinguishable from the uniform distribution, which provides a solid theoretical foundation. The NTRU assumption under a narrow distribution χ is also closely related to Falcon[14], and is sometimes also called the **Decisional Small Polynomial Ratio (DSPR)** assumption[15].

Search Ring-LWR assumption. Let χ be a distribution over R . The advantage of an adversary A in solving the search Ring-LWR problem $\text{RLWR}_{R,q,k,\chi}$ is defined as

$$\text{Adv}_{R,q,k,\chi}^{\text{RLWR}}(A) = \Pr_{a \leftarrow U(R_q^\times), s \leftarrow \chi} \left[A \left(a, \left\lfloor \frac{(as \bmod q)}{k} \right\rfloor \right) = s \right].$$

In our algorithm, $R = \mathbb{Z}[x]/(x^n + 1)$, $R = \mathbb{Z}[x]/(x^n + 1)$, and $\chi = U(R \bmod 2)$.

The theoretical foundation of the search Ring-LWR assumption is developed in [17][18][19]. There are also some practical schemes, such as Lizard [20] and Saber[21], that use Ring-LWR over $\mathbb{Z}[x]/(x^n + 1)$ or its modular variant as their hardness assumption. In fact, the provable hardness of Ring-LWR with binary secrets is still an open problem. However, according to the current state-of-the-art cryptanalytic results, especially when q is relatively small, this does not weaken its concrete security.

KEM security. The security notion proved for the CTL algorithm is IND-CCA security (indistinguishability under chosen-ciphertext attack). To this end, we first note

that the underlying encryption scheme (Algorithms 1, 2, and 3) is IND-CPA secure (indistinguishability under chosen-plaintext attack) under the assumptions in the previous section.

Theorem 1. Let Π be the public-key encryption scheme defined by Algorithms 1, 2, and 3. For any adversary A , there exist adversaries A_1 and A_2 , whose running times are both roughly the same as that of A , such that

$$\text{Adv}_{\Pi}^{\text{IND-CPA}}(A) \leq \text{Adv},$$

where Hash_m is a classical random oracle, and

$$\text{Adv}_{\Pi}^{\text{IND-CPA}}(A) \leq 2d_H \sqrt{\text{Adv}},$$

where A is a quantum adversary, Hash_m is a quantum-accessible random oracle, and d_H is the query depth, where

$$\text{Adv} = \text{Adv}_{R,q,\chi_1}^{\text{NTRU}}(A_1) + \text{Adv}_{R,q,k,\chi_2}^{\text{RLWR}}(A_2),$$

where χ_1 is the distribution of the secret key f, g , and $\chi_2 = U(R \bmod 2)$.

Proof. Let Π_{base} be the public-key encryption scheme defined by Algorithms 1, 2, and 3. We first prove the OW-CPA security of Π_{base} . Let $\Pr[W_i]$ be the probability that the adversary wins in Game i . Game 0 is the classical OW-CPA game, therefore

$$\Pr[W_0] = \text{Adv}_{\Pi_{\text{base}}}^{\text{OW-CPA}}(A).$$

The only difference between Game 1 and Game 0 lies in key generation: in Game 1, the public key h is instead sampled from $U(R_q^\times)$. We use the different distributions of h in Game 0 and Game 1 to construct an adversary A_1 for the decisional NTRU problem. Given input $h \in R_q^\times$, A_1 plays the challenger in Game 0, except that it directly sets $pk = h$; if A wins, then it declares that h is an NTRU public key. Therefore

$$\text{Adv}_{R,q,\chi_1}^{\text{NTRU}}(A_1) = |\Pr[W_0] - \Pr[W_1]|.$$

Next, we define an adversary A_2 for the search Ring-LWR problem. Given input (h, c) , let $pk = h$ and $ct = c$, and A_2 plays the challenger in Game 1. It is easy to see that

$$\text{Adv}_{R,q,k,\chi_2}^{\text{RLWR}}(A_2) = \Pr[W_1].$$

Thus, we immediately obtain

$$\text{Adv}_{\Pi_{\text{base}}}^{\text{OW-CPA}}(A) \leq \text{Adv}.$$

The scheme Π is derived from Π_{base} by using a simple ACWC

transformation[22]. Note that, in the NTRU setting, preimage resistance (PRE-CPA security) is essentially equivalent to OW-CPA security. Combining with (ROM) and (QROM)[22], the above conclusion holds.

CTL_KEM is obtained by applying the Fujisaki–Okamoto transformation [22] to this IND-CPA secure encryption. Theorem 3 gives the concrete-security statement of its IND-CCA security. Theorem 2 is adapted from [22], while these results were first proposed in [23] and [24].

Theorem 2. Let Π be the public-key encryption scheme defined by Algorithms 1, 2, and 3. Let Π_{CCA} be the IND-CCA secure KEM defined by Algorithms 5 and 6. Let M be the message space, δ the decryption error rate, and γ the weakly spread parameter of Π . For any adversary A that makes at most q_D decapsulation queries and q_H hash queries (namely Hash_k and Hash_s), there exists an adversary A' against the IND-CPA security of Π , such that

$$\text{Adv}_{\Pi_{\text{CCA}}}^{\text{IND-CCA}}(A) \leq 2(\text{Adv}_{\Pi}^{\text{IND-CPA}}(A') + \frac{q_H}{|M|}) + q_D \cdot 2^{-\gamma} + q_H \delta,$$

$$\text{Time}(A') \approx \text{Time}(A),$$

where Hash_k and Hash_s are classical random oracles,

$$\text{Adv}_{\Pi_{\text{CCA}}}^{\text{IND-CCA}}(A) \leq 2q_T \sqrt{\text{Adv}_{\Pi}^{\text{IND-CPA}}(A')} + 24q_T^2 \sqrt{\delta} + 24 \sqrt{q_T^3 q_D} \cdot 2^{-\gamma/4},$$

where A is a quantum adversary, and Hash_k and Hash_s are quantum-accessible random oracles, and

$$q_T = 2(q_D + q_H),$$

$$\text{Time}(A') \approx \text{Time}(A) + O(q_H \cdot q_D \cdot \text{Time}(\text{Encrypt}) + q_T^2).$$

Here, the definitions of δ and γ are given in [23] and [24].

6.2 Analysis with respect to known attacks

The security analysis methods for the NTRU algorithm can, in terms of objective, be divided into key-recovery attacks and message-recovery attacks.

Key-recovery attack. The goal of a key-recovery attack is to recover the private key from public information (such as the public key). In NTRU, the public key is a polynomial, while the private key consists of two relatively short polynomials f and g , and their product modulo a large integer q and modulo the polynomial ring R yields the public key. A lattice attacker hopes to find the

private-key polynomials f and g by constructing an appropriate lattice and exploiting the short-vector property of that lattice.

Specifically, the key-recovery attack constructs a lattice for NTRU, called the NTRU lattice. This lattice is usually a matrix structure containing private-key information, with a specific size and shape. Given the public key $h = f^{-1} * g \pmod{q}$, one can construct a basis matrix whose column vectors form a lattice, where the matrix has the form

$$B = \begin{pmatrix} qI & 0 \\ H & I \end{pmatrix}$$

Here, I is the identity matrix, and H is the convolution matrix generated from the public-key polynomial h . In this lattice, the private-key vector (f, g) is regarded as a relatively short vector. The attacker can use lattice basis reduction algorithms (such as LLL or BKZ) to search for short vectors in the lattice. If the short vector found is sufficiently short and satisfies the private-key properties of NTRU, then the attacker can successfully recover the private key. After finding a short vector, it is further necessary to verify whether this vector can recover the original private-key polynomials f and g . If the vector found satisfies the specific NTRU equations, then the key-recovery attack succeeds.

Message-recovery attack. For the encryption algorithm in this paper, the ciphertext is generated by convolving the plaintext polynomial with the public-key polynomial and reducing modulo q . The strategy of a message-recovery attack is to find a way to recover the original plaintext polynomial from the ciphertext. For a given ciphertext $c = r * h + m \pmod{q}$, where r is the random polynomial generated during encryption, m is the plaintext, and h is the public key, the attacker can construct a lattice related to the message-recovery problem. Then, they try to find a short vector in this lattice that corresponds to the plaintext or a polynomial closely related to it. Lattice basis reduction algorithms such as LLL or BKZ are used to search for short vectors. Since the plaintext is usually a short polynomial, the algorithm may find a relatively short vector in the lattice basis, thereby corresponding to the decryption of the ciphertext. After finding the short vector, check whether the vector matches the expected structure of the plaintext (such as bit length or polynomial coefficient range), and verify its correctness according to

the decryption formula. If the verification succeeds, then the message-recovery attack succeeds.

Primal attack. Consider the NTRU problem with modulus q , the degree n , the standard deviation of the coefficient distribution χ_f of f is σ_f , and the standard deviation of the distribution χ_g of g is σ_g . Given the public key $h = f^{-1} * g \pmod{q}$, one can construct a basis matrix whose column vectors form a lattice, where the matrix has the form

$$B = \begin{pmatrix} qI & 0 \\ H & I \end{pmatrix}$$

where I is the identity matrix, and H is the $n \times n$ convolution matrix generated from the public-key polynomial h . In this lattice, the private-key vector (f, g) is regarded as a relatively short vector, and this vector can be solved by lattice basis reduction algorithms.

Hybrid attack. When carrying out a message-recovery attack against our scheme, the vector that needs to be solved is relatively short. For such a structure, the hybrid attack method is usually effective. The strategy of a hybrid attack is to first guess part of the secret key, and then seek a balance between the guessing cost and the lattice basis reduction cost. Among guessing techniques, the meet-in-the-middle (MITM) method proposed in [11] performs best in terms of complexity.

Dual attack: Consider the decisional version of the problem $\text{LWE}_{q,n,m,\chi_s,\chi_e}$. The Dual method [12] first transforms the LWE problem into an SIS problem, and then uses the solution of the SIS problem to transform solving the LWE problem into distinguishing distributions under certain parameters. Its basic idea is that, given a set of instances $(A, b = As + e) \in \mathbb{Z}_q^{m \times n} \times \mathbb{Z}_q^m$, consider the dual lattice $\Lambda_q^\perp = \{x \in \mathbb{Z}^m | xA = 0 \pmod{q}\}$ generated by A . Use lattice basis reduction algorithms to find a short vector in $\Lambda_q^\perp$. The resulting vector v then satisfies $vA = 0$, that is, it solves the SIS problem. Compute $\langle v, b \rangle = \langle v, e \rangle \pmod{q}$, and determine the distribution of $\langle v, e \rangle$. Since e follows a Gaussian distribution, $\langle v, b \rangle$ also follows a Gaussian distribution over $\mathbb{Z}_q$ (typically, v is a short vector, while e is a small error, so $\langle v, b \rangle$ is also a relatively small number); if b is chosen uniformly at random, then $\langle v, b \rangle$ is also uniformly random over $\mathbb{Z}_q$. Therefore, the vector obtained by solving the SIS problem can be used to distinguish these two cases, thereby solving the decisional-LWE

problem.

6.3 Decryption Failure Probability

Let polynomial ring $\mathcal{R} = \mathbb{Z}[x]/(x^n + 1)$, where the degree n is a power of two and q denotes a positive integer modulus. Polynomials $f, g \in \mathcal{R}$ follow the discrete Gaussian distribution over integers $D_{\mathbb{Z}, \sigma_f}$ independently and identically, and $h = f^{-1}g \pmod{q}$. The NTRU trapdoor basis is defined as $\mathbf{B}_{f,g} = \begin{pmatrix} g & G \\ f & F \end{pmatrix}$. Random variables s and e are sampled independently from distributions χ_s and χ_e respectively, with $\mu_e, \mu_s \in \frac{1}{Q}\mathbb{Z}$. Define the standard deviation ratio $\gamma = \sigma_e/\sigma_s$ and positive integer q' , and compute auxiliary vector $w = \text{ComputeVec}(\mathbf{B}_{f,g}, q', \gamma)$.

The auxiliary function is given by $I(\gamma, r, R) = \frac{\ln(R^2/r^2)}{\gamma(R^2 - r^2)}$, where the inner and outer radii of the annular sampling region satisfy $r = \left(\frac{1}{3}\alpha + \frac{2}{3\alpha}\right)\sqrt{q}$, $R = \left(\frac{2}{3}\alpha + \frac{1}{3\alpha}\right)\sqrt{q}$. The equivalent standard deviations are formulated as $\sigma_1 = \sqrt{n} \cdot \sigma_f(\alpha, q, n) \sigma_s \cdot \sqrt{\gamma^2 + 1}$, $\sigma_2 = \sigma_s \left(\frac{q\gamma\sqrt{I(\gamma, \alpha, q)}}{\sqrt{n} \cdot \sigma_f(\alpha, q, n)} + \frac{n^{1.5} \cdot \sqrt{\gamma^2 + 1} \cdot \sigma_f(\alpha, q, n)}{2q'} \right)$.

Set threshold parameter $\tau = \min\left(\frac{q}{2\sqrt{2}\sigma_1}, \frac{q}{2\sqrt{2}\sigma_2}\right)$. The upper bound of decoding failure probability for the Decode algorithm is $P_{\text{dec}} \leq 2n \cdot (1 - \text{erf}(\tau))$. Each coefficient of γs and e is mutually independent, leading to the combined standard deviation $\sigma_{\text{se}} = \sqrt{\gamma^2 \sigma_s^2 + \sigma_e^2} = \sqrt{(k^2 - 1)/6}$. The upper bound of failure probability caused by norm overflow is $P_{\text{chk}} \leq 2n \cdot \text{erfc}(1.08/\sqrt{2})$. Consequently, the heuristic upper bound for the overall decryption failure probability is expressed as:

$$P_{\text{fail}} \leq 2n[1 - \text{erf}(\tau)] + 2n \left[1 - \text{erf}\left(\frac{1.08}{\sqrt{2}}\right) \right].$$

An increment in decryption failure probability linearly raises the adversary's winning advantage under chosen-ciphertext attacks and impairs the provable IND-CCA security of the scheme. Two types of failure outcomes originating from core decoding and norm verification introduce dual statistical leakage; once the failure probability exceeds the safety threshold, adversaries can launch statistical CCA attacks and hybrid key-recovery attacks by analyzing the distribution of successful and failed decryption results. Reducing the overall failure probability via tuning modulus q and annular

sampling parameter α improves the underlying NTRU lattice’s resistance against lattice reduction attacks, whereas aggressive parameter compression weakens lattice security margin. Frequent decryption failures induce more conditional branch jumps during implementation and aggravate side-channel leakage via power consumption and traffic fingerprinting. Benefiting from the dual-restriction design, the proposed scheme reserves extra security redundancy compared with conventional counterparts.

7. Implementation and Performance Evaluation

7.1 Performance Analysis

(1) High-quality Generation of (f, g) via Hybrid Sampler

This paper adopts an NTRU lattice hybrid sampler to constrain trapdoor quality at the distribution design level. Leveraging the Fourier embedding structure of the cyclotomic quotient ring, near-uniform sampling is performed in the frequency domain together with a rounding-error compensation mechanism, such that sampled outputs inherently satisfy the geometric constraints of the trapdoor basis. This approach eliminates abundant invalid samples and improves sampling success rate while suppressing fluctuation of trapdoor quality and enhancing overall algorithm stability.

(2) Optimization of Judgment Criteria and Workflow for NTRU Equation Solving

Since solving the NTRU equation constitutes the dominant computational bottleneck during key generation, pre-verification criteria are proposed on the premise of improved (f, g) sampling quality to filter out infeasible samples prior to equation computation. An adaptive iteration termination rule is further formulated to eliminate redundant iterative loops and cut unnecessary resource consumption, thereby boosting the overall efficiency of key generation.

(3) Iteration Strategy and Optimization for NTRU Equation

To tackle the drawbacks of conventional NTRU solvers including low iteration efficiency, excessive stack footprint and uncontrollable failure probability, tailored iterative optimizations are proposed for the targeted application scenario. The

ReducePartial iteration strategy is deployed to strike a trade-off among iteration rounds, solving failure probability and computational speed and reduce the aggregate solving failure rate. A lightweight intermediate verification mechanism matching the features of Reduce is constructed to detect invalid iterations with negligible overhead and terminate them in advance, effectively reducing useless computational cost.

7.2 Performance Testing

This chapter focuses on the engineering implementation of the CTL algorithmic scheme and presents its practical feasibility and efficiency characteristics. The evaluation adopts the average number of CPU cycles over 1000 independent runs as the principal performance metric, thereby quantifying the end-to-end computational cost of key generation, encoding/decoding, encryption/decryption, and encapsulation/decapsulation under the three parameter sets, namely CTL-128, CTL-256, and CTL-512. These results provide a quantitative basis for the engineering deployment of the proposed algorithm.

All experiments were conducted on a Lenovo Legion R9000P ARX8 laptop. The hardware configuration includes an AMD Ryzen 9 7945HX processor (16 cores, 32 threads, base clock of 2.5 GHz, maximum boost clock of 5.4 GHz), dual-channel Samsung DDR5 5200 MHz memory, a 1 TB UMIS NVMe SSD, and an NVIDIA RTX 4060 Laptop GPU with 8 GB GDDR6 video memory, running the 64-bit Windows 11 Professional operating system.

The code is written in the C programming language. The compilation environment adopts a GCC-compatible C compiler and GNU Make build tools. Compilation and linking depend on the GMP library (libgmp), the Quadmath library (libquadmath), and the standard mathematics library (libm). The algorithm is implemented based on a self-developed cryptographic framework. The highest optimization level is enabled during compilation. The average CPU cycle count (in cycles) from 1000 independent runs is adopted as the quantitative performance metric. The test results are presented in Table

5. For the lengths of public keys, private keys and ciphertexts, see Section 3.5.

Table 5 Performance evaluation results

Algorithm	Dimension n	Key Generation-a	Key Generation-b	Encaps	Decaps
CTL-128	512	11621k	12023k	24190	175566
CTL-256	1024	46158k	48166k	54215	368216
CTL-512	2048	1503798k	2991984k	5583134	41486048

Note: Key Generation-a denotes the key generation performance adopting the optimized NTRU equation solving method proposed in this paper; Key Generation-b represents the key generation performance of the conventional unoptimized NTRU equation solving method.

After optimization with the AVX2 instruction set and assembly, the code achieves approximately 10% performance improvement compared with the unoptimized version.

Table 6 Performance Test Results After Optimization

Algorithm	Dimension n	Key Generation	Encaps	Decaps
CTL-128	512	10343k	18590	114613
CTL-256	1024	38620k	29792	208848
CTL-512	2048	1423102k	94662	42912604

8. Conclusion

This paper addresses several limitations of the traditional NTRU algorithm, including inefficient trapdoor generation, substantial computational bottlenecks in key solving, and insufficient coverage of security parameter levels, and proposes three principal improvements.

First, a high-quality generation method based on a hybrid sampler is introduced.

By abandoning the traditional and inefficient sample-and-filter paradigm, this method relies on the Fourier embedding structure of cyclotomic rings to perform sampling in the frequency domain, together with an error-compensation mechanism, so as to impose direct constraints on trapdoor quality. This design not only reduces the number of invalid samples and improves both the sampling success rate and algorithmic stability, but also supports more flexible parameter configurations.

Second, the paper optimizes the decision criteria and execution procedure for solving the NTRU equation. By incorporating a preliminary validity test, designing an adaptive iteration termination strategy, and employing an efficient solving approach, the proposed method alleviates the principal computational bottleneck in key generation, reduces average computational complexity, and improves the overall efficiency of key generation. As such, it is more suitable for resource-constrained environments.

Third, the work redesigns the multi-level security parameter system. By extending the set of high-security parameter configurations and uniformly optimizing the core parameters across all security levels, the proposed scheme achieves a more favorable balance among algorithmic security, computational efficiency, and decryption failure probability.

References

- [1] Shor P W .Polynomial-Time Algorithms for Prime Factorization and Discrete Logarithms on a Quantum Computer[J].Siam Review, 1999, 41(2):303-332.DOI:10.1137/S0097539795293172.
- [2] Grover L K. A fast quantum mechanical algorithm for database search[C]/Proceedings of the twenty-eighth annual ACM symposium on Theory of computing. 1996: 212-219.
- [3] Avanzi R, Bos J, Ducas L, Kiltz E, Lepoint T, Lyubashevsky V, Schanck JM, Schwabe P, Seiler G, Stehlé D. CRYSTALS-Kyber: Algorithm specifications and supporting documentation (version 3.01). 2021. <https://pq-crystals.org/kyber/data/kyber-specification-round3-20210131.pdf>

- [4] Hoffstein J, Pipher J, Silverman J H. NTRU: A ring-based public key cryptosystem[C]//International algorithmic number theory symposium. Berlin, Heidelberg: Springer Berlin Heidelberg, 1998: 267-288.
- [5] Ajtai M. Generating hard instances of lattice problems[C]//Proceedings of the twenty-eighth annual ACM symposium on Theory of computing. 1996: 99-108.
- [6] Fouque P A, Kirchner P, Pornin T, et al. BAT: Small and fast KEM over NTRU lattices[J]. IACR Transactions on Cryptographic Hardware and Embedded Systems, 2022: 240-265.
- [7] Espitau T, Nguyen T T Q, Sun C, et al. Antrag: Annular NTRU trapdoor generation: making mitaka as secure as falcon[C]//International Conference on the Theory and Application of Cryptology and Information Security. Singapore: Springer Nature Singapore, 2023: 3-36.
- [8] Pornin T. Improved (Again) Key Pair Generation for Falcon, BAT and Hawk[J]. Cryptology ePrint Archive, 2025.
- [9] Coppersmith D, Shamir A. Lattice attacks on NTRU[C]//International conference on the theory and applications of cryptographic techniques. Berlin, Heidelberg: Springer Berlin Heidelberg, 1997: 52-61.
- [10] Stehlé D, Steinfeld R. Making NTRU as secure as worst-case problems over ideal lattices[C]//Annual international conference on the theory and applications of cryptographic techniques. Berlin, Heidelberg: Springer Berlin Heidelberg, 2011: 27-47.
- [11] May A. How to meet ternary LWE keys[C]//Annual International Cryptology Conference. Cham: Springer International Publishing, 2021: 701-731.
- [12] Micciancio D, Regev O. Lattice-based Cryptography[J]. Post-Quantum Cryptography, 2009, 1: 147.
- [13] Stehlé D, Steinfeld R. Making NTRU as secure as worst-case problems o

- ver ideal lattices[C]//Annual international conference on the theory and applications of cryptographic techniques. Berlin, Heidelberg: Springer Berlin Heidelberg, 2011: 27-47.
- [14] Fouque P, Hoffstein J, Kirchner P, et al. Falcon: Fast-Fourier Lattice-based Compact Signatures over NTRU[J]. Post-Quantum Cryptography Standard, 2020.
 - [15] López-Alt A, Tromer E, Vaikuntanathan V. On-the-fly multiparty computation on the cloud via multikey fully homomorphic encryption[C]//Proceedings of the forty-fourth annual ACM symposium on Theory of computing. 2012: 1219-1234.
 - [16] Brakerski Z, Döttling N. Lossiness and entropic hardness for ring-LWE[C]//Theory of Cryptography Conference. Cham: Springer International Publishing, 2020: 1-27.
 - [17] Banerjee A, Peikert C, Rosen A. Pseudorandom functions and lattices[C]//Annual international conference on the theory and applications of cryptographic techniques. Berlin, Heidelberg: Springer Berlin Heidelberg, 2012: 719-737.
 - [18] Bogdanov A, Guo S, Masny D, et al. On the Hardness of Learning with Rounding over Small Modulus[C]//Proceedings, Part I, of the 13th International Conference on Theory of Cryptography-Volume 9562. 2016: 209-224.
 - [19] Chen L, Zhang Z, Zhang Z. On the hardness of the computational Ring-LWR problem and its applications[C]//International Conference on the Theory and Application of Cryptology and Information Security. Cham: Springer International Publishing, 2018: 435-464.
 - [20] Cheon J H, Kim D, Lee J, et al. Lizard: Cut off the tail! A practical post-quantum public-key encryption from LWE and LWR[C]//International conference on security and cryptography for networks. Cham: Springer Inter

national Publishing, 2018: 160-177.

- [21] Vercauteren I F, Roy S S, D’Anvers J P, et al. SABER: Mod-LWR based KEM (round 3 submission)[J]. ProQuest Number: INFORMATION TO ALL USERS, 2020, 31657053.
- [22] Duman J, Hövelmanns K, Kiltz E, et al. A thorough treatment of highly-efficient NTRU instantiations[C]//IACR International Conference on Public-Key Cryptography. Cham: Springer Nature Switzerland, 2023: 65-94. Dent, A.W.: A designer’s guide to KEMs. In: IMA International Conference on Cryptography and Coding. pp. 133–151. Springer (2003)
- [23] Hofheinz D, Hövelmanns K, Kiltz E. A modular analysis of the Fujisaki-Okamoto transformation[C]//Theory of Cryptography Conference. Cham: Springer International Publishing, 2017: 341-371.
- [24] Don J, Fehr S, Majenz C, et al. Online-extractability in the quantum random-oracle model[C]//Annual International Conference on the Theory and Applications of Cryptographic Techniques. Cham: Springer International Publishing, 2022: 677-706.